



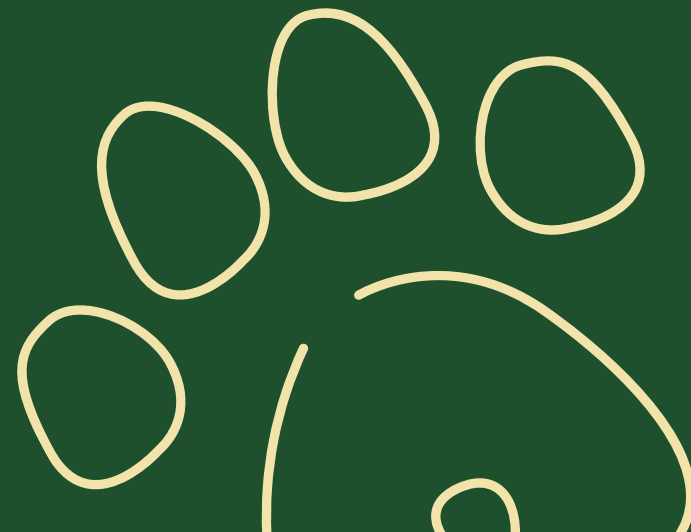
WOOFWALKS

PRESENTATION DE DOSSIER



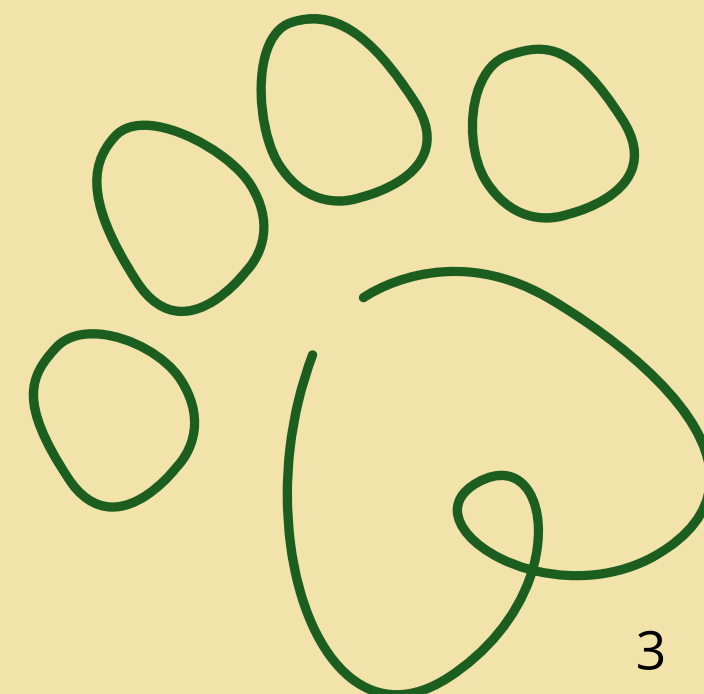
SOMMAIRE

- I. Présentation du service
- II. Expressions des besoins, contraintes, livrables
- III. Gestion de projet
- IV. Spécifications fonctionnelles
- V. Captures d'écran et extraits de code
- VI. Éléments de sécurité
- VII. Plans de tests
- VIII. Jeux d'essai
- IX. Veille
- X. Synthèse et conclusion

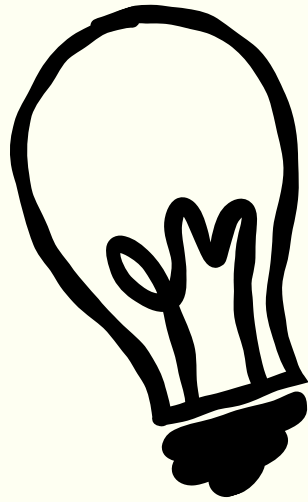




I. PRÉSENTATION DU PROJET



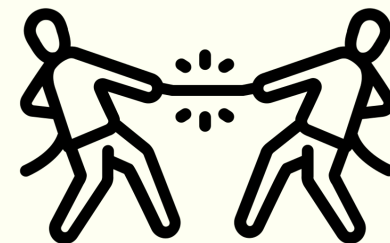
I.1 UTILITÉ ET CONCURRENCE



Organisation de balades

Référencement de parcs

Référencement de randonnées ...

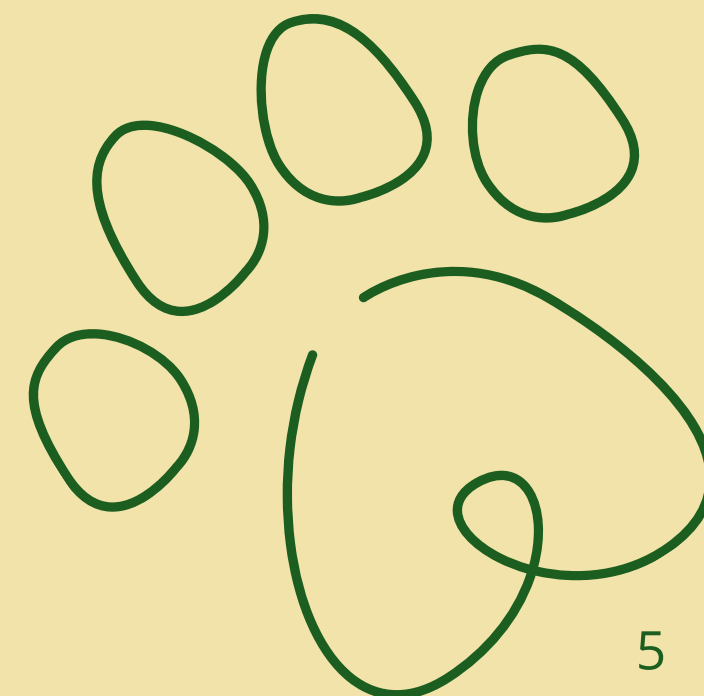


Concurrence

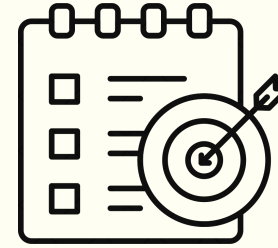
Groupe Facebook locaux, sites et applications peu fréquentées



II. EXPRESSIONS DES BESOINS, CONTRAINTES, LIVRABLES



II.1 OBJECTIFS DU PROJET



Réseau social visant à :

- Lutter contre l'isolement des propriétaires
- Favoriser la socialisation des chiens
- Référencer des lieux et activités adaptées



Public cible

Les propriétaires de chiens de tout niveau social et tout âge



II.2 CONTRAINTES LIVRABLES

- 🎨 Design attractif
- ⚡ Rapidité
- 🧭 Ergonomie
- 📖 Organisation claire
- ✍️ Contenu de qualité
- 🔒 Sécurité
- 📄 Documentation





II.3 CHARTE GRAPHIQUE

Le logo :



Design:

Les polices, couleurs répondent aux exigences du **WCAG**.
L'ensemble est **responsive** et respecte les **normes d'accessibilités**

Police: Utilisation d'une police adaptée comme : **Lexend**, Atkinson Hyperlegible, Open Sans, Roboto

Élément	Desktop	Tablette	Mobile
Texte de corps	16px – 18px	16px	16px
Titres principaux (h1)	32px – 36px	28px – 32px	24px – 28px
Titres secondaires (h2, h3)	24px – 28px	20px – 24px	18px
Boutons et liens	18px - 20px	18px -20px	20px

Voici un exemple de police Lexend, premier choix paramétré en CSS pour des raisons de lisibilité

Element	Couleur	Vue
Fond	Blanc-gris	#D9D9D9
Boutons, Menu Burger, Header	Vert forêt	#1B5E20
Texte boutons	Blanc cassé	#F5F5F5
Logo et Bouton Burger	Blanc	#FFFFFF
Texte général	Noir	#000000
Annulations, Erreurs	Rouge	#FF0000



WCAG Compliance Results		
ELEMENT TYPE	AA	AAA
Small Text	✓ Pass	✓ Pass
Large Text	✓ Pass	✓ Pass

IV.4 LES UTILISATEURS ET LEURS DROITS

Visiteur



Utilisateur Loggé





II.5 INVENTAIRE DES BESOINS FONCTIONNELS

Thème	Qui	Quoi	Pourquoi
Compte	Visiteur	M'inscrire	Accéder aux fonctionnalités de l'application et participer aux
Authentification	Utilisateur	Me connecter	Utiliser la totalité des fonctionnalités de l'application
Authentification	Utilisateur	Me déconnecter	Quitter l'application
Compte	Utilisateur	Modifier mes informations de profil	Mettre à jour mes données personnelles et afficher les
Localisation	Utilisateur	Modifier ma localisation à un instant t	Adapter les suggestions de balades à ma position actuelle
Compte	Utilisateur	Supprimer mon compte	Quitter la plateforme et effacer mes données personnelles
Notifications	Utilisateur	Choisir mes préférences de notifications	Recevoir uniquement les informations qui m'intéressent
Animal	Utilisateur	Ajouter les informations sur mon chien	Pour que les autres utilisateurs puissent avoir une idée de la
Contact	Visiteur Utilisateur	Contacter les administrateurs de l'application	Communiquer avec l'entreprise, pour toute information
Modération	Utilisateur	Signaler un utilisateur ou du contenu non approprié ou	Enclencher un contrôle afin de garantir un espace sûr
Annonce	Utilisateur	Créer une annonce	Préparer une annonce pour sortir mon chien avec d'autres maîtres,
Annonce	Utilisateur	Poster une annonce	Publier définitivement l'annonce créée
Annonce	Utilisateur	Modifier une annonce	Ajuster l'annonce, prévenir les inscrits d'un changement grâce à
Annonce	Utilisateur	Supprimer une annonce	Empêcher de nouvelles inscriptions et prévenir les inscrits
Annonce	Utilisateur	Poster une copie d'une ancienne annonce	Gagner du temps si j'organise souvent les mêmes balades
Lieu	Utilisateur	Créer une annonce dans un lieu référencé comme parc	Proposer un lieu connu et adapté aux chiens
Lieu	Utilisateur	Créer une annonce dans un lieu customisé	Organiser une balade dans un endroit moins connu ou privé

Thème	Qui	Quoi	Pourquoi
Annonces	Utilisateur	Afficher toutes les annonces autour de moi	Visualiser rapidement l'ensemble des balades prévues susceptibles de me
Annonces	Utilisateur	Filtrer les annonces selon mes critères	Trouver plus rapidement une balade qui correspond à mes envies
Annonces	Utilisateur	Trier les annonces selon mes critères	Trouver plus rapidement une balade qui correspond à mes envies
Participants	Utilisateur	Consulter le profil des participants et de leurs chiens	Évaluer la compatibilité entre les animaux, voir si cela me correspond
Feedbacks	Utilisateur	Consulter les textes des feedbacks des anciennes	Évaluer la qualité des événements passés et savoir si cela correspond à
Annonces	Utilisateur	Voir les détails d'une annonce	Savoir précisément où, quand et comment elle se déroule
Balades	Utilisateur	Voir qui est inscrit à la balade	Avoir une idée des participants et de l'ambiance prévue
Balades	Utilisateur	Voir les balades prévues sur une carte	Visualiser rapidement où elles ont lieu
Balades	Utilisateur	M'inscrire à une balade	Participer à une balade qui me correspond
Balades	Utilisateur	Annuler mon inscription	Libérer ma place et permettre à quelqu'un d'autre de s'inscrire
Notifications	Utilisateur	Demander à recevoir des notifications si une place se	Être alerté pour pouvoir m'inscrire dès qu'une place est disponible
Album	Utilisateur	Sélectionner des photos des balades auxquelles j'ai assisté	Sélectionner celles qui me plaisent le plus
Album	Système	Valider l'achat de mon album photo	Recevoir l'album que j'ai créé
Chat	Utilisateur	Accéder au chat de la balade dès inscription	Échanger avec les autres participants de la balade
Chat	Utilisateur	Envoyer des messages aux utilisateurs inscrits dans la	Échanger avec les autres participants avant la balade
Chat	Utilisateur	Recevoir des messages des utilisateurs de la balade	Être informé en direct et pouvoir réagir
Notifications	Utilisateur	Être notifié des changements de la balade	Être immédiatement au courant des évolutions



II.6 PRODUCT BACKLOG : LISTE DES USER STORIES

Coefficients planning poker	Durée (Jours)
1	0,5
2	1
3	1,5
4	3
5	5

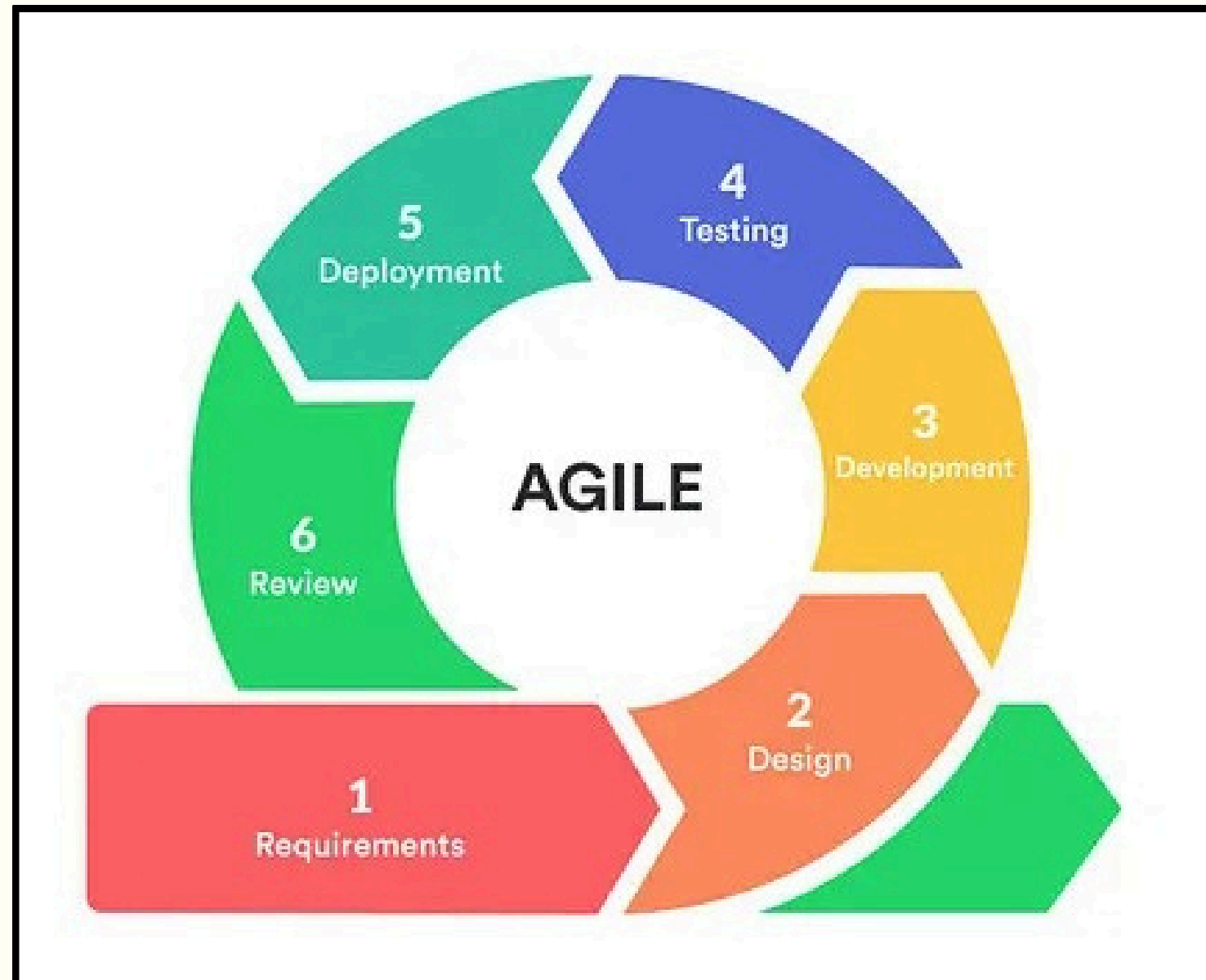
En tant que ...	Je veux...	Afin de ...	Priorité	PlanningPoker Remits 1/5	Commentaire	Durée estimée
Utilisateur	Choisir mes préférences de notifications	Recevoir uniquement les informations qui m'intéressent sans être submergé	C		Mise en place d'un formulaire « configuration du système de notifications de l'utilisateur »	1,5
Acteur et Utilisateur	Consulter les administrateurs de l'application	Communiquer avec l'entreprise, pour toute information	C		Mise en place d'un formulaire de contact	1
Utilisateur	Signaler un utilisateur ou du contenu non approprié ou contraire aux RQPD	Endoscher un conseil afin de garantir un espace sûr	C		Mise en place d'un formulaire de contact	1
Utilisateur	Poster une annonce qui est une copie d'une annonce	Gagner du temps si j'organise souvent les mêmes balades	C		Mise en place d'un formulaire simple	1
Utilisateur	Créer une annonce dans un lieu spécifique comme parc	Proposer un lieu connu et adapté aux chiens	C		État dans la page Création d'une annonce	5
Utilisateur	Consulter le profil des participants et leurs chiens	Évaluer la compatibilité entre les animaux, voir si cela me correspond	C		« Trouvé en Etop »	1
Utilisateur	Consulter les notes des feedback des anciennes balades de l'organisateur	Évaluer la qualité des événements passés et savoir si cela correspond à mes attentes	C		« Trouvé en Etop »	1
Utilisateur	Voir qui est invité à la balade	Afin d'avoir une idée des participants et de l'ambiance prévue	C		3	1,5
Utilisateur	Avoir une vue sur une carte geo des balades prévues	S'assurer rapidement où sont les balades prévues	C		Mise en place carte interactive leaflet et positionnement des lieux selon la localisation des annonces en BDD	1
Utilisateur	Créer un album photo	Sélectionner les photos qui me plaisent le plus pour faire mon album	C		« Trouvé Etop » - Requêtes complexes en première LIX facile	5
Utilisateur	Voir toutes les annonces autour de moi	S'assurer rapidement l'ensemble des balades prévues susceptibles de me correspondre	C		Filter selon localisation actuelle de l'utilisateur (modifiable à tout moment: voir dans « Créer et gérer un compte ») - Nécessite l'implémentation du service de géolocalisation « Trouvé en Etop »	1,5
Système	Valider l'achat de mon album photo	Recevoir l'album que j'ai créé	C		Multitenant, panier, paiement, retour, gestion erreurs, envoi confirmation, enregistrement commande en BDD	5
Utilisateur invité à une balade	Recevoir des notifications de tout changement de l'annonce	Être immédiatement au courant si l'heure, le lieu ou d'autres détails évoluent	C		« ... nécessite la mise en place du système de notifications	1
Utilisateur invité à une balade	Recevoir des notifications du chat	Recevoir informé des échanges entre participants	C		« ... nécessite la mise en place du système de notifications	1
Utilisateur invité à une balade	Recevoir des notifications lorsqu'une place dans la balade se libère	Pouvoir m'inscrire avant que d'autres ne prennent la place	C		« ... nécessite la mise en place du système de notifications	1
Utilisateur invité à une balade	Envoyer des feedback contenant des photos, du texte et une note globale	Partager mon expérience et alimenter la banque de photo	C		Mise en place d'un formulaire avec texte et photos	1
Utilisateur invité à une balade	Consulter les feedback de la balade à laquelle j'ai été	Voir les retours des autres participants et revoir l'événement à travers leurs photos et commentaires	C		2	1
Acteur	M'inscrire	Accéder aux fonctionnalités de l'application et participer aux balades	M		MCréation de compte avec (pseudo, mot de passe, photo, localisation, et vérification par mail (ou autre méthode)	5
Utilisateur	Créer une annonce dans un lieu spécifique (api geo)	Proposer un lieu connu et adapté aux chiens	M		1	1,5
Utilisateur	Me connecter	Utiliser la totalité des fonctionnalités de l'application	M		3	1,5
Utilisateur	Me déconnecter	Quitter l'application	M		2	1
Utilisateur	M'inscrire à une balade	Participer à une balade qui me correspond	M		2Simple click sur bouton « Etop » confirmation par email	1
Administrateur	Consulter toutes les annonces publiées	Surveiller leur conformité	M		1	0,5
Administrateur	Recevoir des notifications des utilisateurs signalant du contenu inapproprié	éditer le contenu inapproprié	M		2	1
Administrateur	Accéder et modifier les profils des utilisateurs	Modifier les comptes en cas de demande explicite d'un utilisateur ou fournir un utilisateur si besoin	M		3	1,5
Administrateur	Supprimer un compte utilisateur	Répondre à la demande d'un utilisateur ou fournir un utilisateur qui ne respecterait pas les RQPD	M		2	1
Administrateur	Moderer les message dans les Chats des balades	éditer le contenu inapproprié	M		2	1
Administrateur	Moderer les feedbacks	éditer le contenu inapproprié	M		2	1
Utilisateur	Annuler mon inscription	Libérer ma place si je ne peux plus participer et permettre à quelqu'un d'autre de s'inscrire	N		1	0,5
Utilisateur	Voir les détails d'une annonce	Savoir précisément où, quand et comment elle se déroule avant de m'inscrire	N		4	1
Utilisateur	Modifier mes informations de profil	Mettre à jour mes données personnelles et afficher les bonnes informations aux autres utilisateurs	N		Mise en place d'un formulaire simple avec photo - Nécessite la mise en place de la récupération des données de l'api geo.gov	1
Utilisateur	Modifier ma localisation à un instant t - depuis toutes les pages	Adapter les suggestions de balades à ma position actuelle	N		4 ou par acceptation de géolocalisation	1
Utilisateur	Supprimer mon compte	Quitter la plateforme et effacer mes données personnelles	N		Mise en place d'un formulaire simple - traitement à faire en conformité avec les RQPD	1,5
Utilisateur	Ajouter les informations sur mon chien	Les autres utilisateurs pourront avoir une idée de la taille, comportement, tempérament... de mon chien	N		Mise en place d'un formulaire simple	1
Utilisateur	Modifier une annonce	1. Ajouter l'annonce car elle ne correspond pas ou plus exactement à l'événement tel qu'il a été guidé 2. Prévenir les inscrits d'un changement de programme grâce au déplacement d'une alerte	N		« Précédé » mise en place du système de notifications Mise en place d'un formulaire simple, s'il envoie d'éventuelles notifications	5



III. GESTION DE PROJET



III.1 MÉTHODOLOGIE



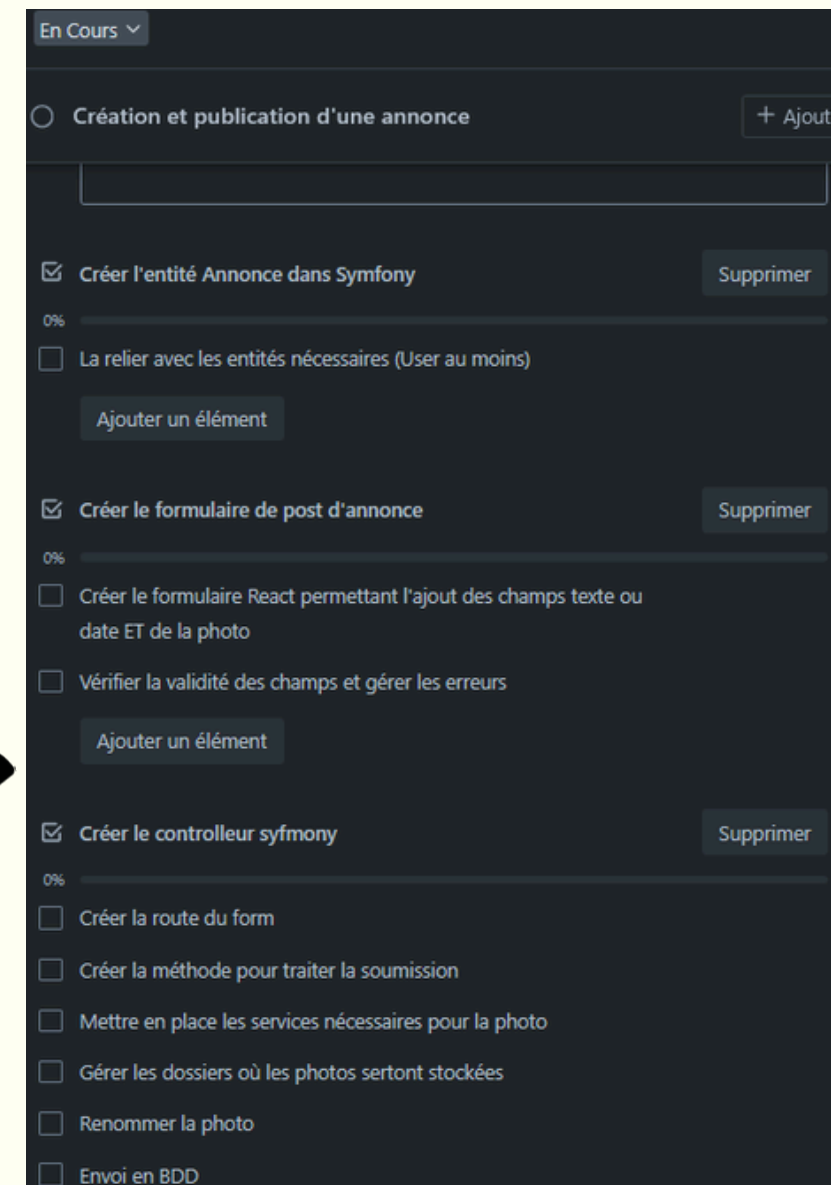
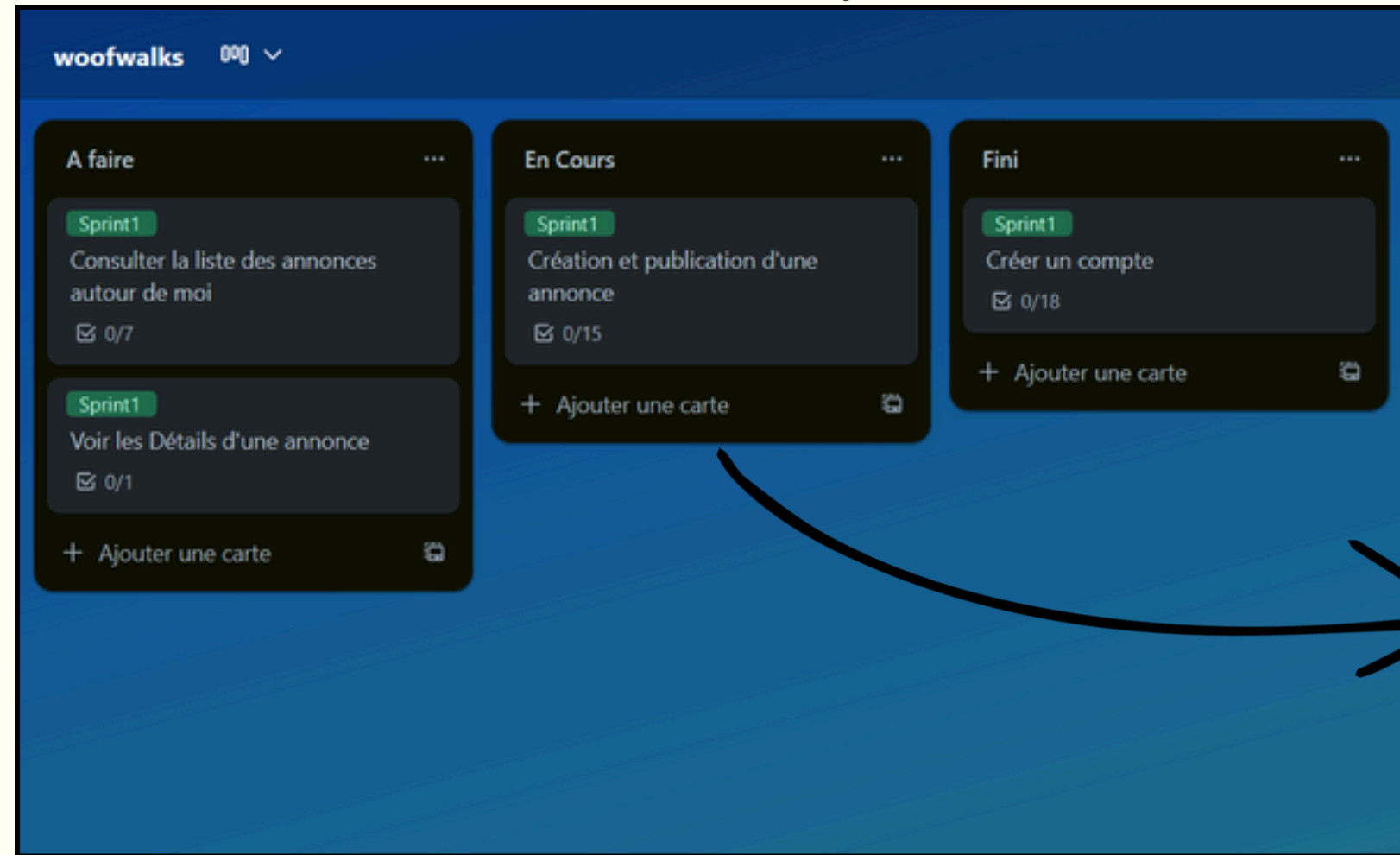
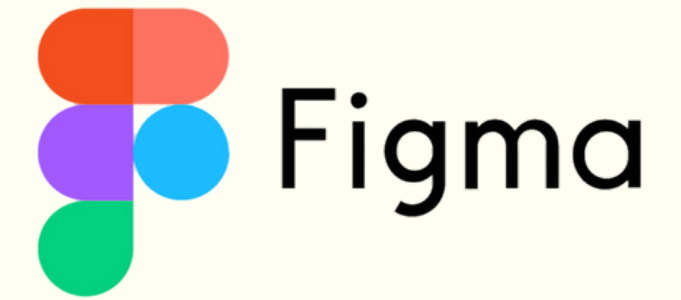
- **Itérations courtes**
- **User stories**
- **Planning poker**
- MoSCoW
- Suivi avec Trello

👉 **Adaptabilité & feedback client**





III.2 OUTILS



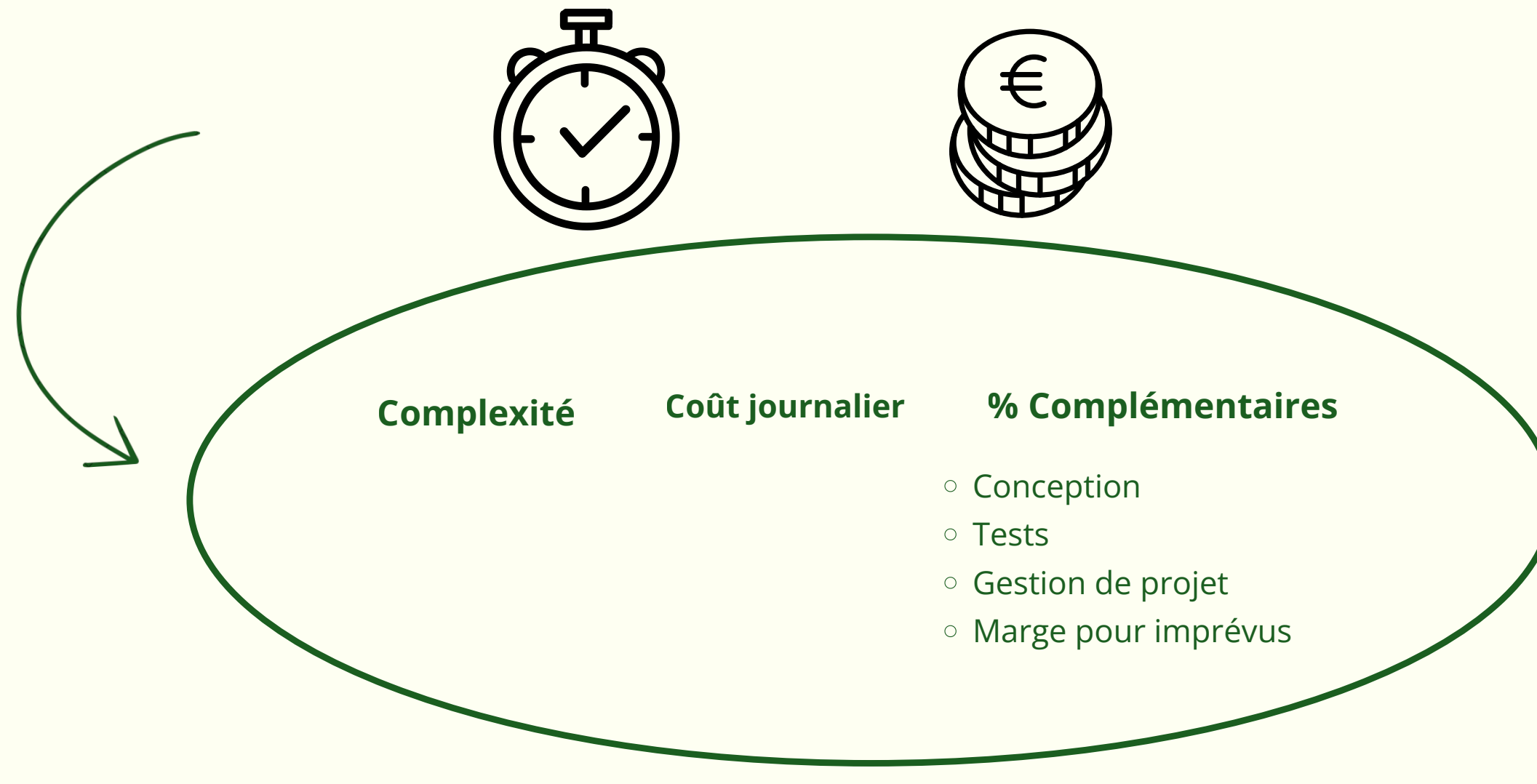
III.3 BUDGET

Priorisation

MVP

- Niveau 1 : fonctionnalités indispensables
- Niveau 2 : fonctionnalités importantes mais non vitales
- Niveau 3 : fonctionnalités complémentaires ou d'optimisation

Chiffrage





IV. SPÉCIFICATIONS FONCTIONNELLES

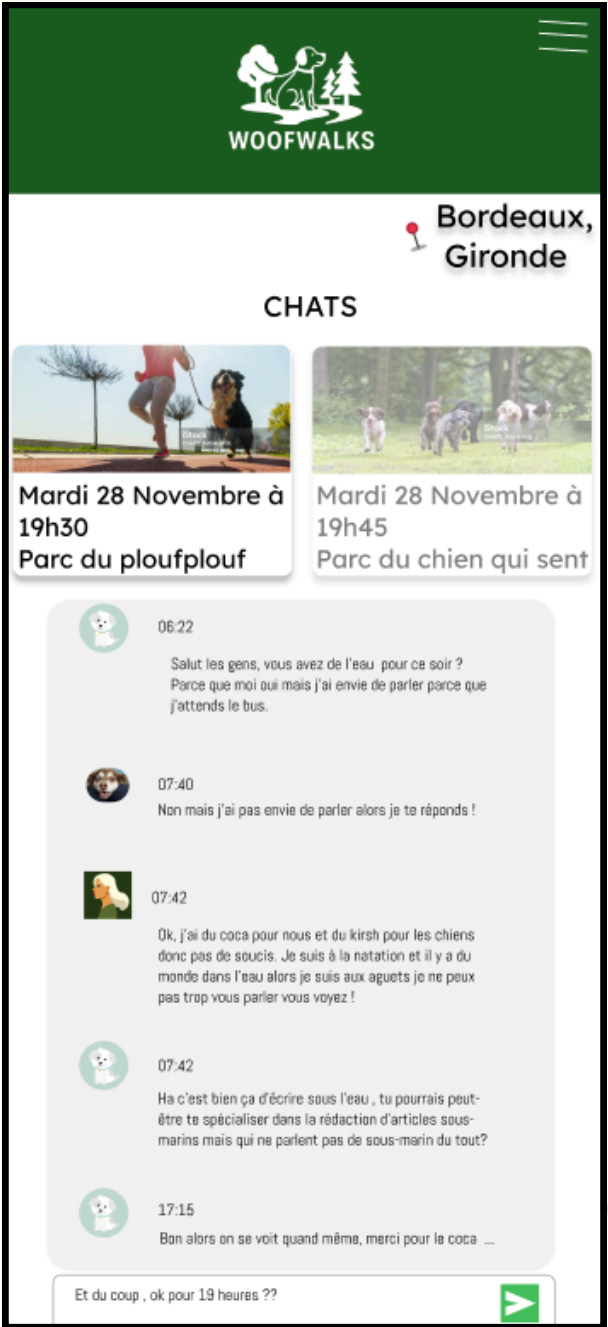
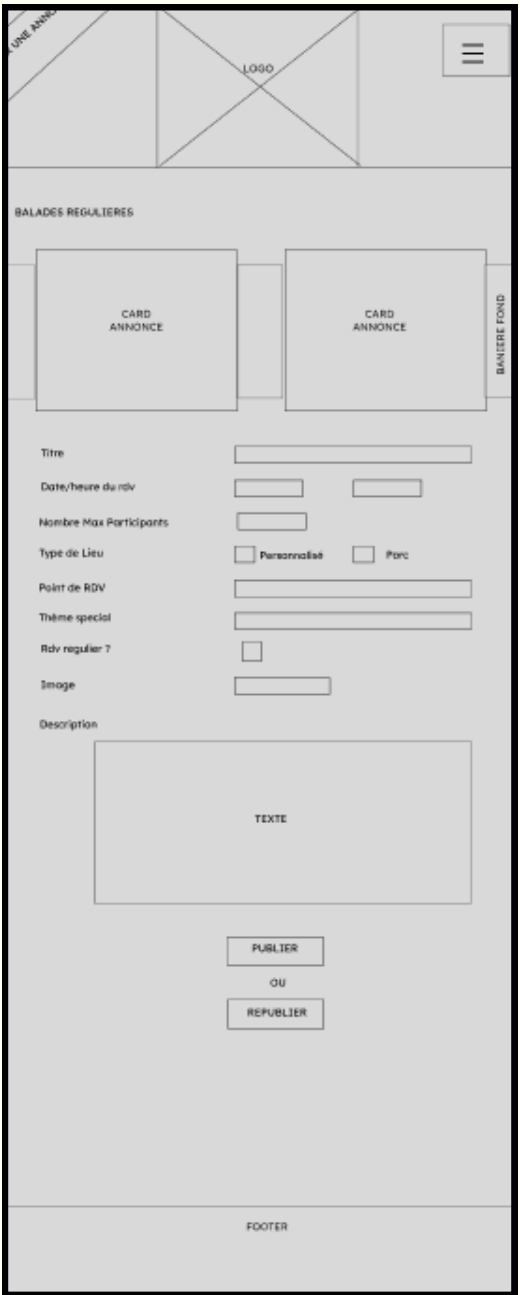
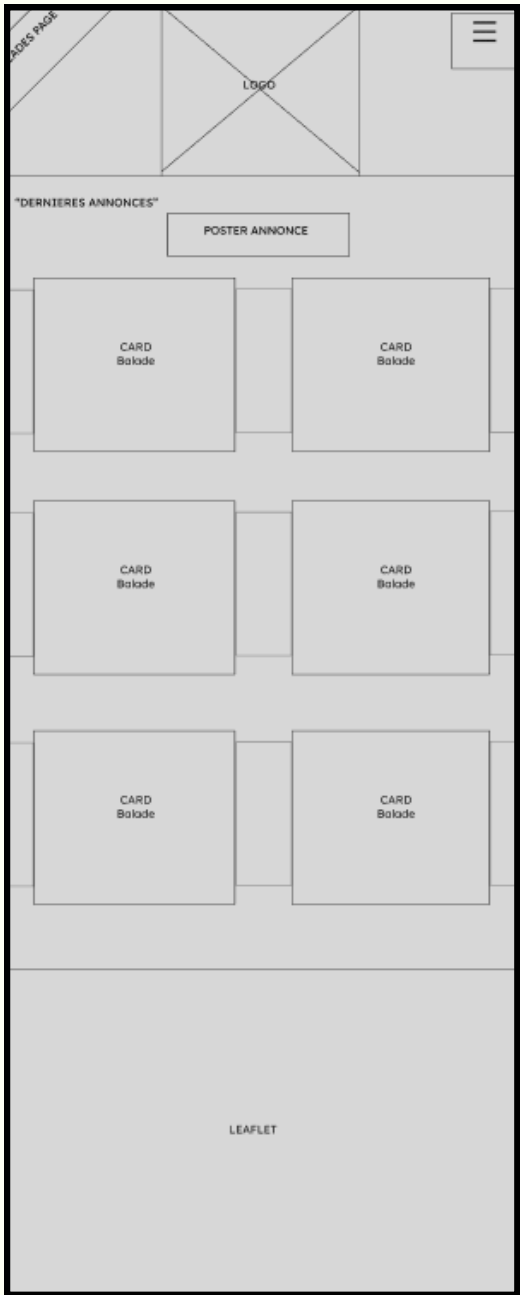


IV.I CARTOGRAPHIE

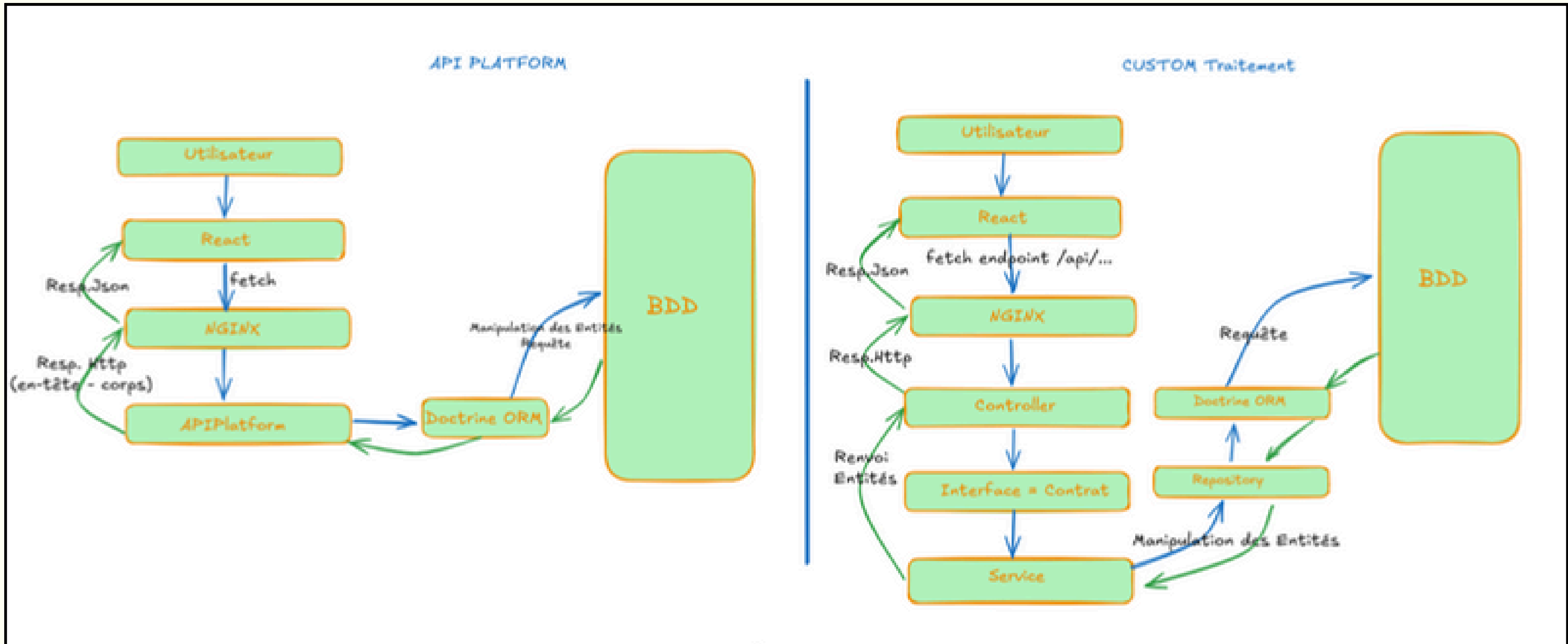




IV.2 WIREFRAMES ET MAQUETTES

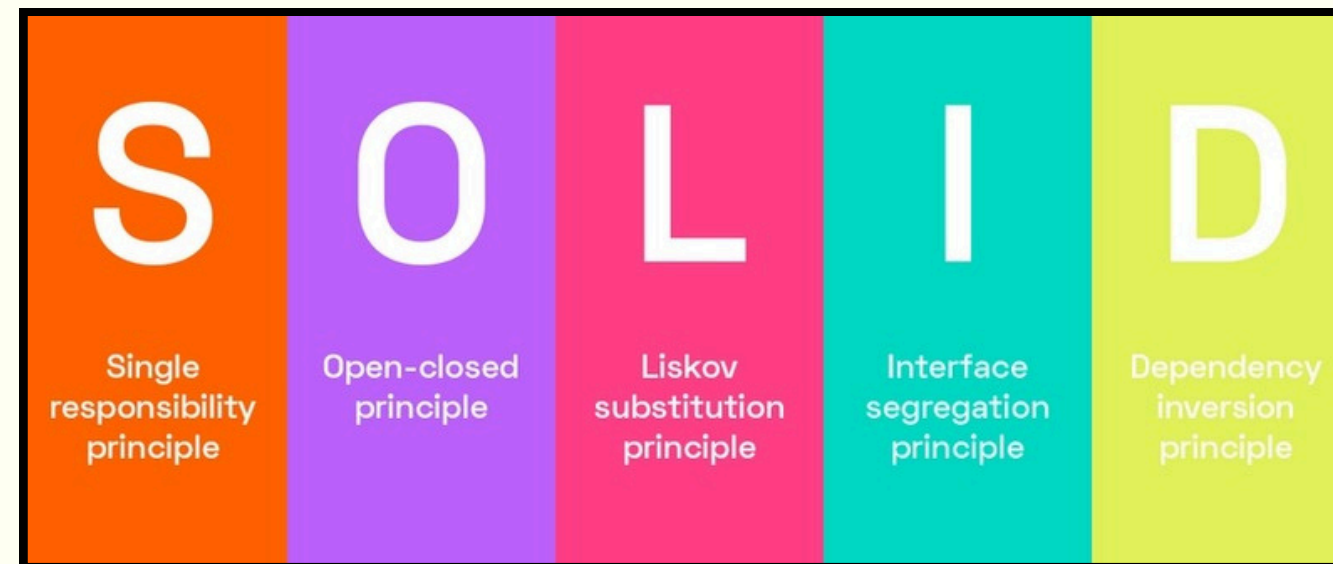


IV.3 ARCHITECTURE MULTI-COUCHES

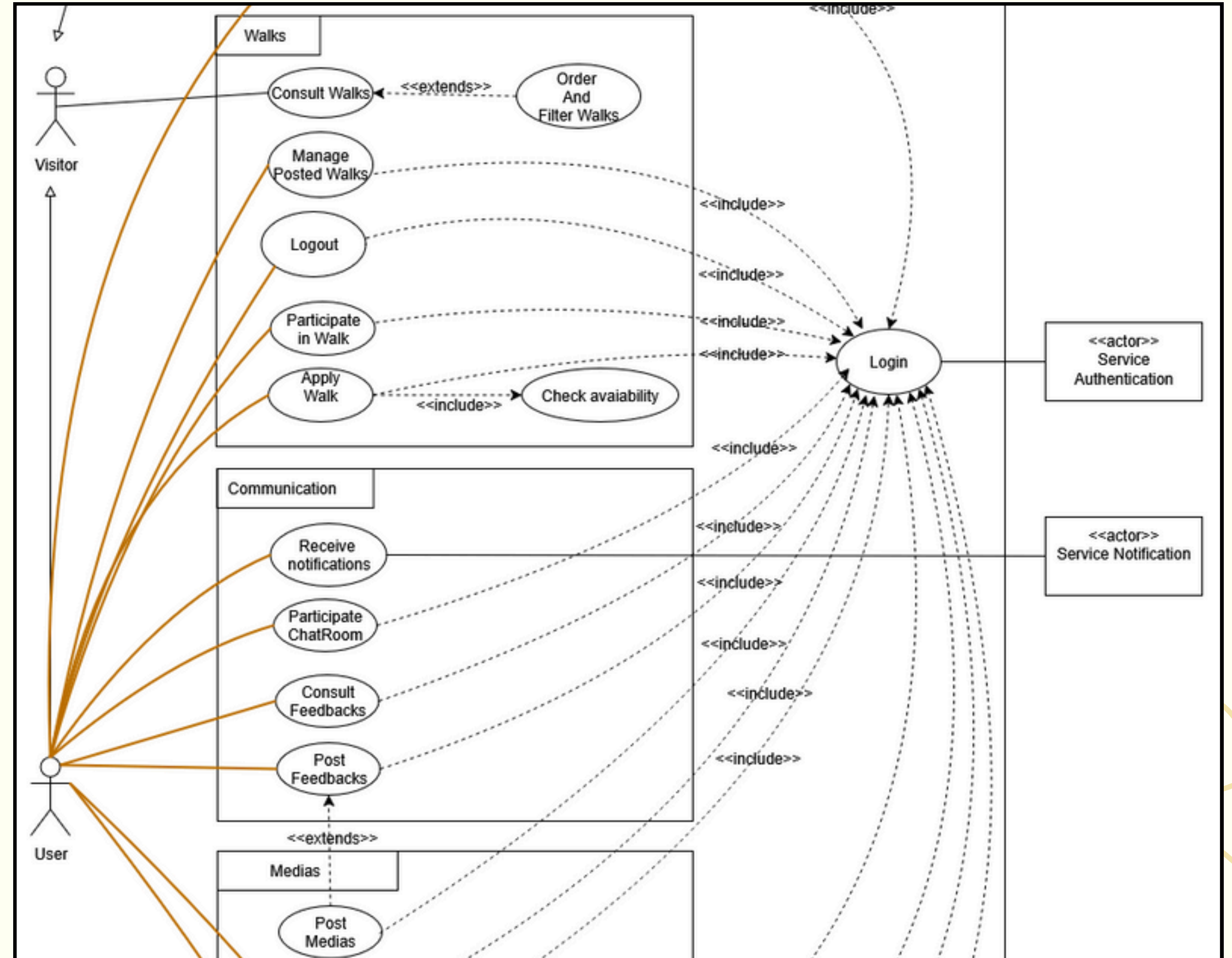
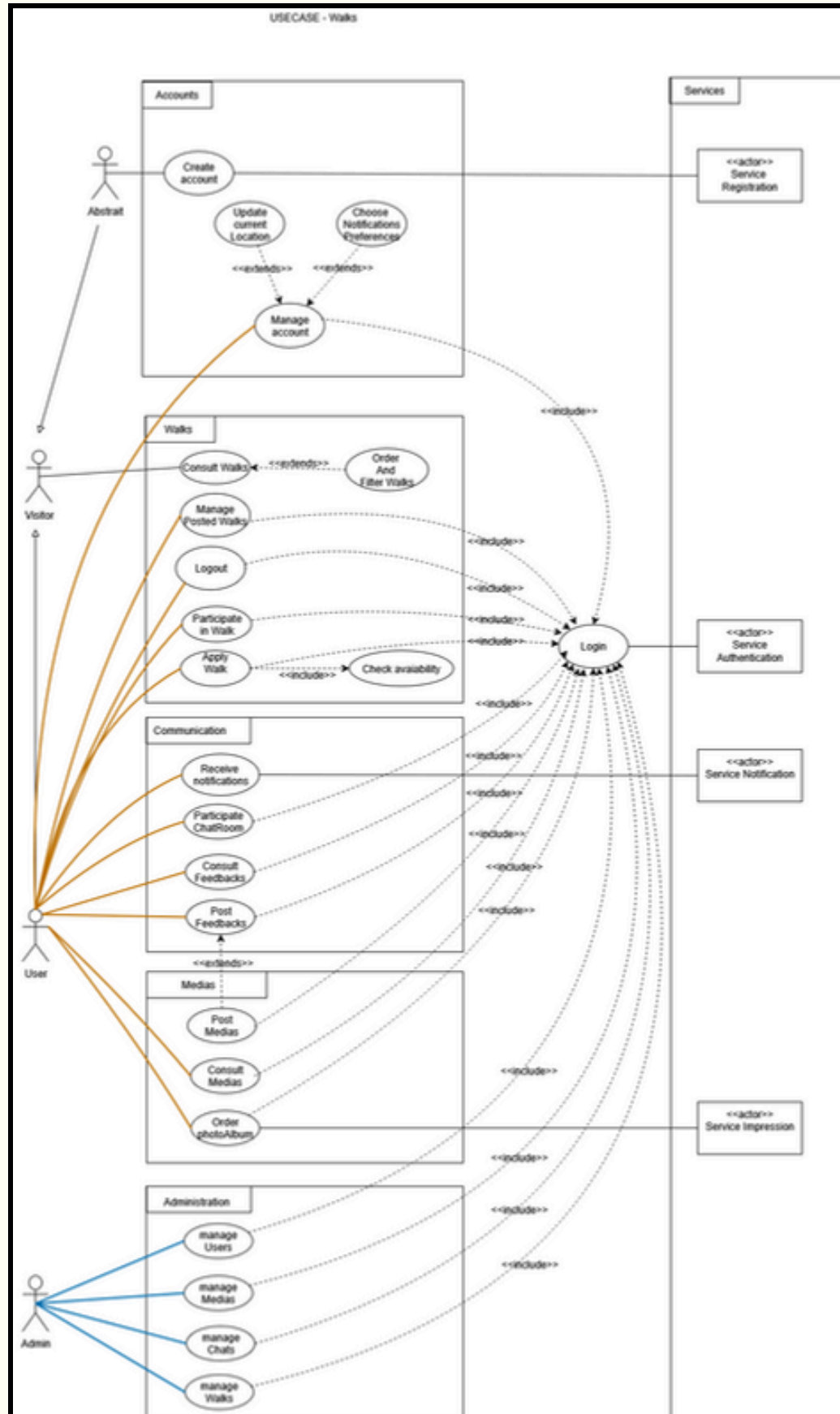


IV.4 PATTERNS ET COMPOSANTS MÉTIER

- Repository Pattern
- Observer Pattern (Events Listeners)
- Service Layer
- Context Pattern (AuthContext)
- Orchestration Pattern
- Validation Pattern
- Interfaces

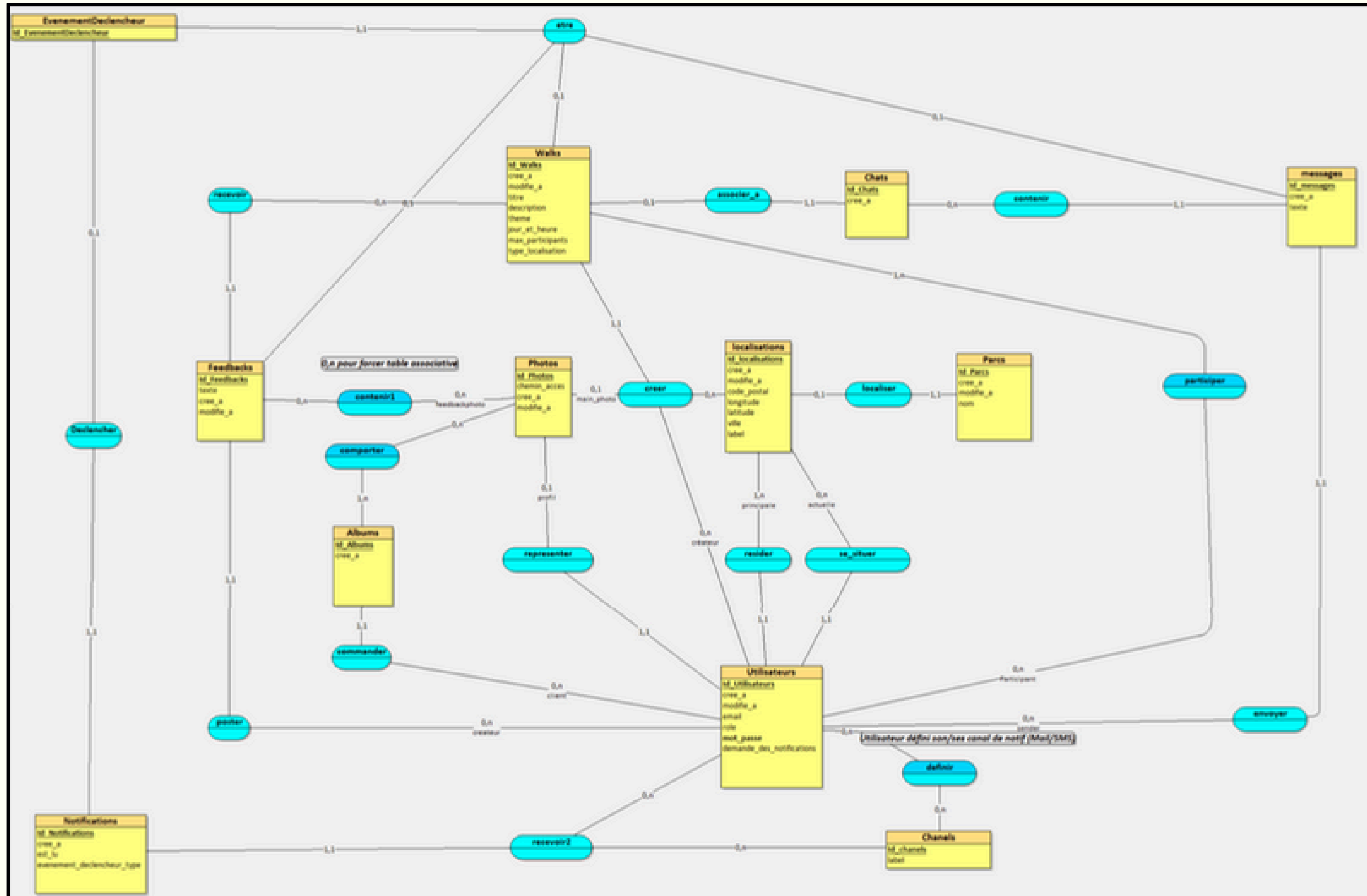


IV.10 UML : USE CASE

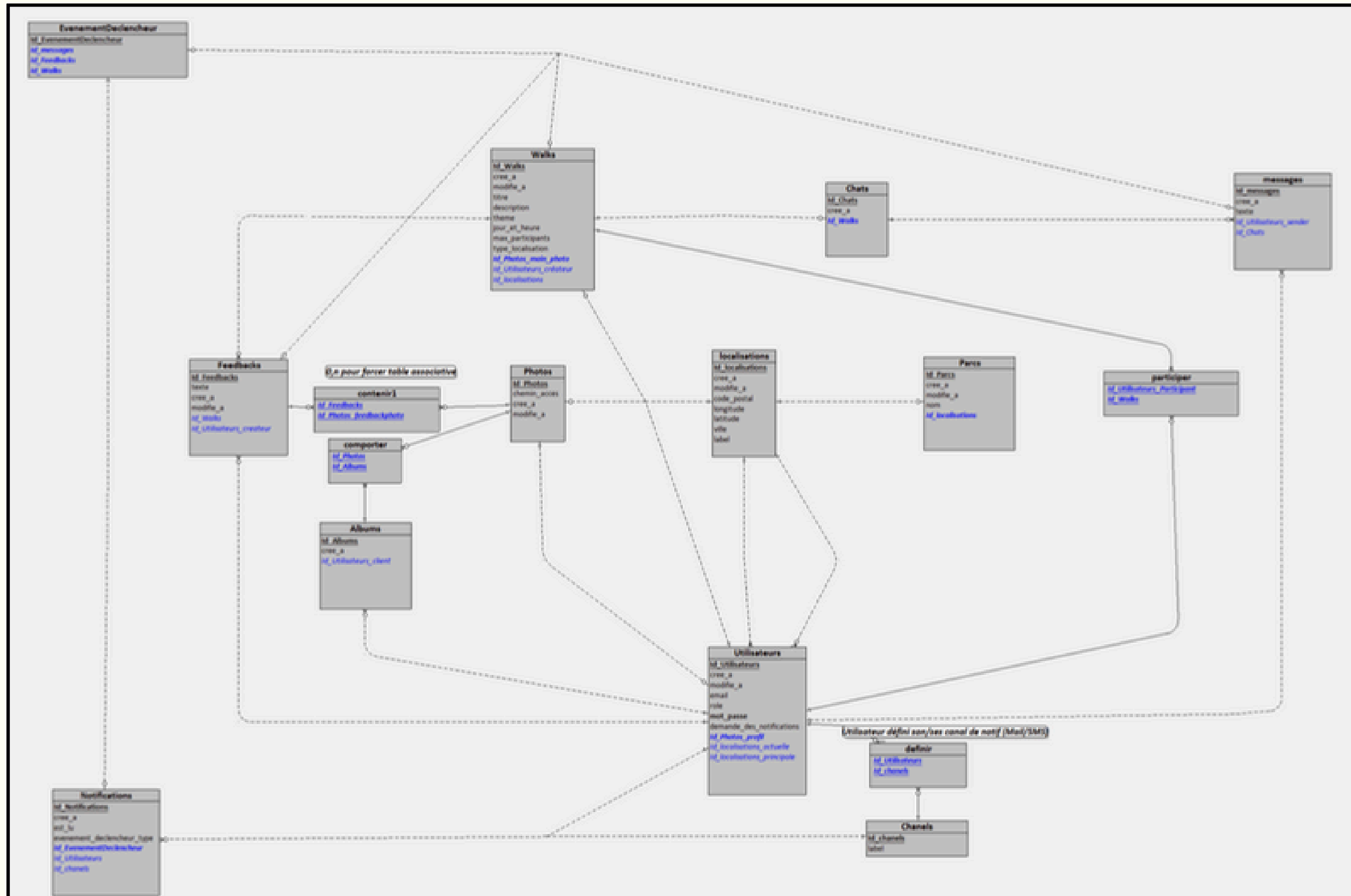




IV.5 MERISE : MODÈLE CONCEPTUEL DE DONNÉES



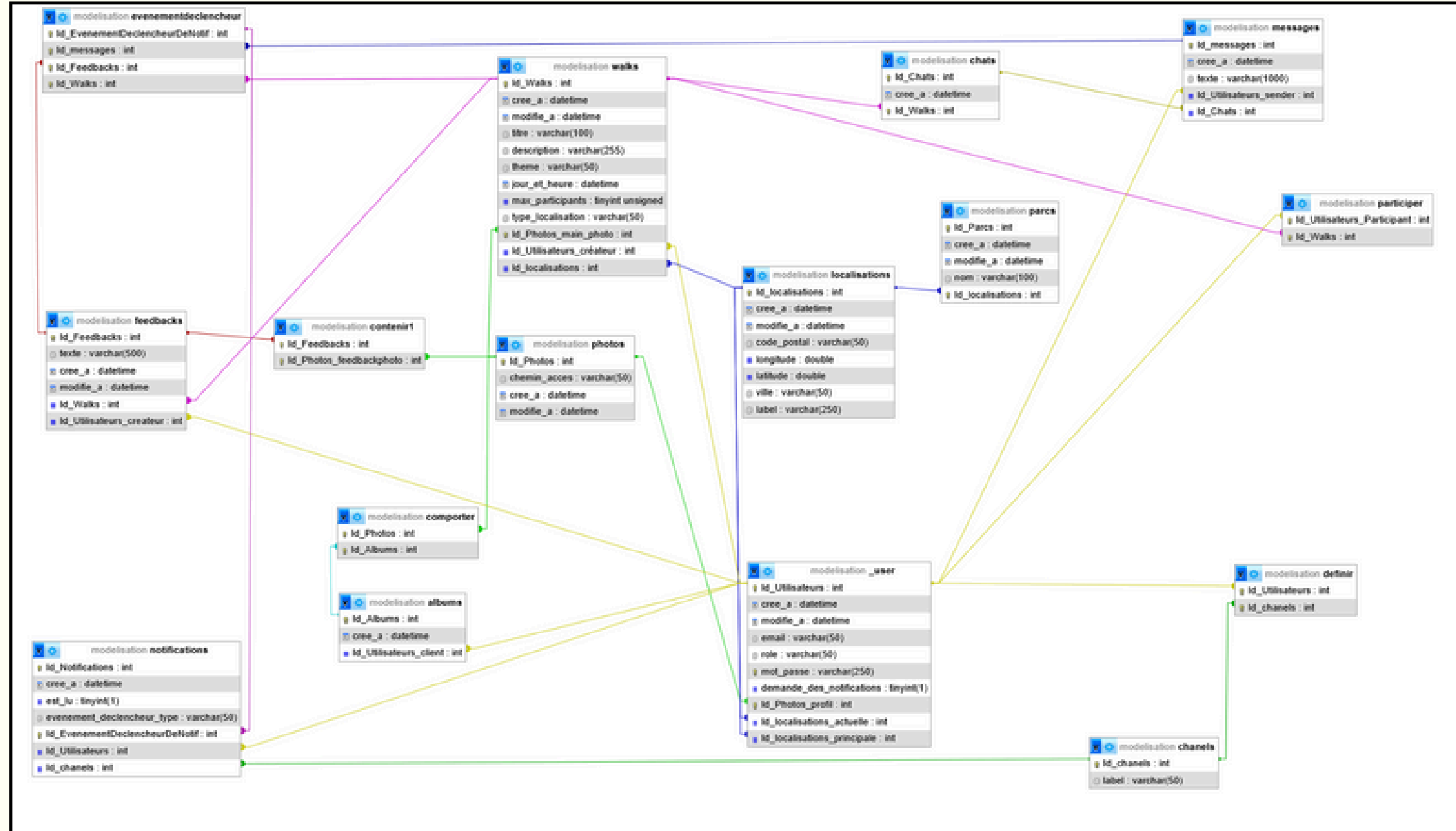
IV.6 MERISE : MODÈLE LOGIQUE DE DONNÉES

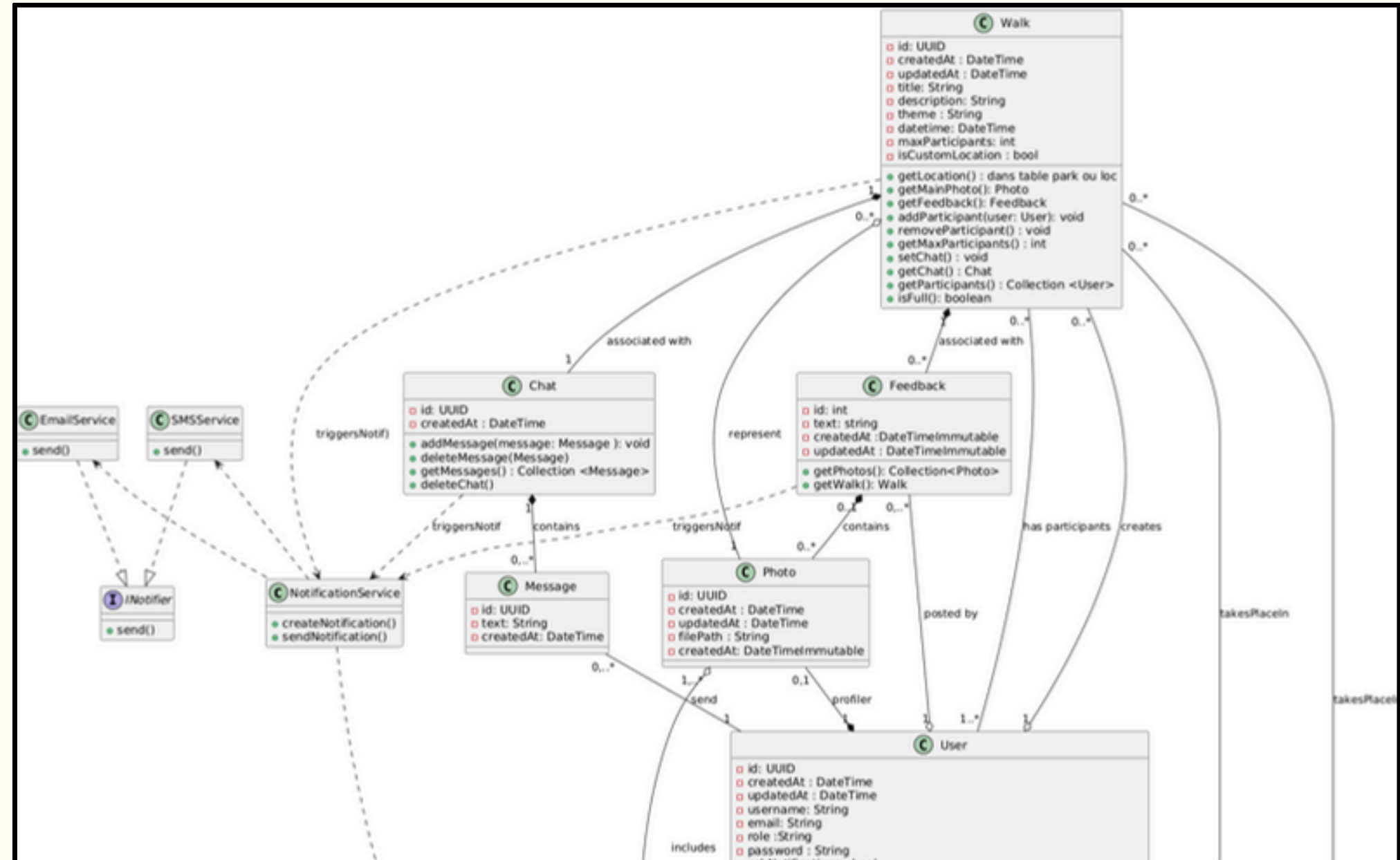


IV.7 DICTIONNAIRE DE DONNÉES

Code Mnémonique	Type	Désignation	Taille	Remarques
User				
d_utilisateur	N	Identifiant unique de l'utilisateur	5	
cree_a	DATE	Heure de la création		
nom_utilisateur	A	Nom de l'utilisateur	100	
role	A	Rôle	50	
demande_notifications	B	Demande des notifications		
email	A	Adresse email de l'utilisateur	150	
mot_de_passe	A	Mot passe de l'utilisateur	250	
Walk				
d_walk	N	Identifiant unique de la promenade	5	
cree_a	DATE	Date de création de la promenade		
modifie_a	DATE	Heure de la modification		
date_et_heure_balade	DATE	Heure de la promenade		
titre	N	Titre de la promenade	2	
description	A	Contenu de la promenade	500	
theme	A	Thème spécifié	100	Peut être NULL
max_participants	N	Nombre maximum de participants	3	Parc ou custom pour chercher dans la bonne table
type_localisation	A	Type de localisation		
Feedback				
d_feedback	N	Identifiant unique du Feedback	5	
cree_a	DATE	Date de création du Feedback		
texte_feedback	A	Contenu du Feedback	500	
Photo				
d_photo	N	Identifiant unique de la photo	5	
chemin_fichier	AN	Chemin de la photo	255	
Album				
d_album	N	Identifiant unique de l'album	5	
cree_a	DATE	Date de création de l'album		
Chat				
d_chat	N	Identifiant unique du chat	5	
cree_a	DATE	Date de création du Chat		
ChatMessage				
d_message	N	Identifiant unique du message	5	
cree_a	DATE	Date de création du message		
texte	A	Contenu du message	255	
Album				
cree_a	DATE	Date de création de l'album		
Evenement_declencheur				
d_evenement_declencheur	N	Identifiant unique de l'événement déclencheur de notification	5	Exemple : nouveau message chat Ou annonce modifiée...
Notification				
d_notification	N	Identifiant unique de la notification	5	
cree_a	DATE	Date de création		
est_lu	B	La notification est lue	50	
Chanel				
d_channels	N	Identifiant unique du chanel	5	
label	A	Libellé du chanel	50	
Parc				
d_parc	N	Identifiant unique du parc	5	
label	A	Libellé du parc	50	
cree_a	DATE	Date de création du Parc		
modifie_a	DATE	Heure de la modification		
Localisation				
d_localisation	N	Identifiant unique du chanel	5	
label	A	Libellé de la localisation	50	
cree_a	DATE	Date de création de la localisation		
modifie_a	DATE	Heure de la modification		
longitude	N	Longitude de la localisation	250	
latitude	N	Latitude de la localisation	250	
code_postal	AN	Code Postal de la localisation	50	
ville	AN	Ville de la localisation	250	

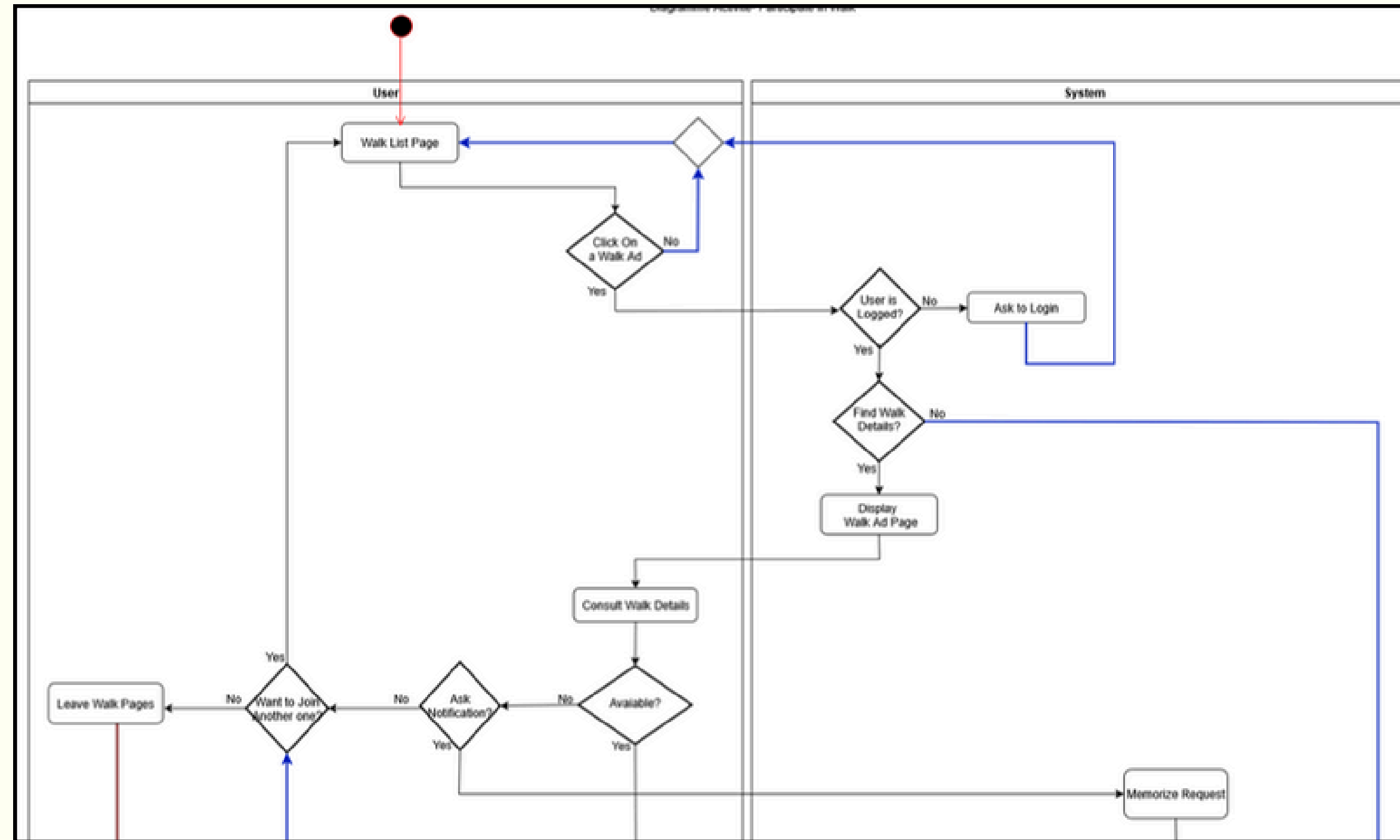
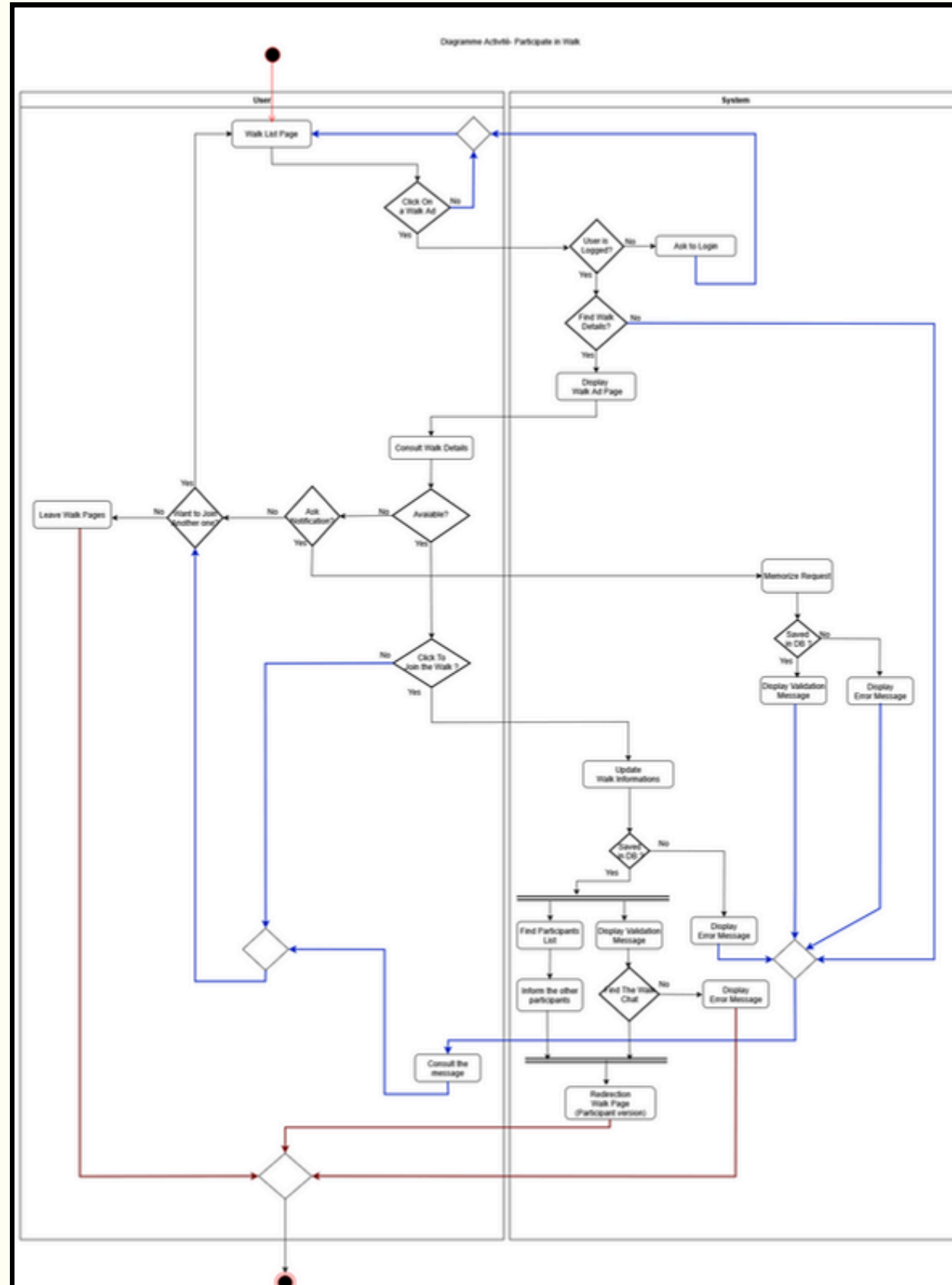
IV.8 MERISE : MODÈLE PHYSIQUE DE DONNÉES





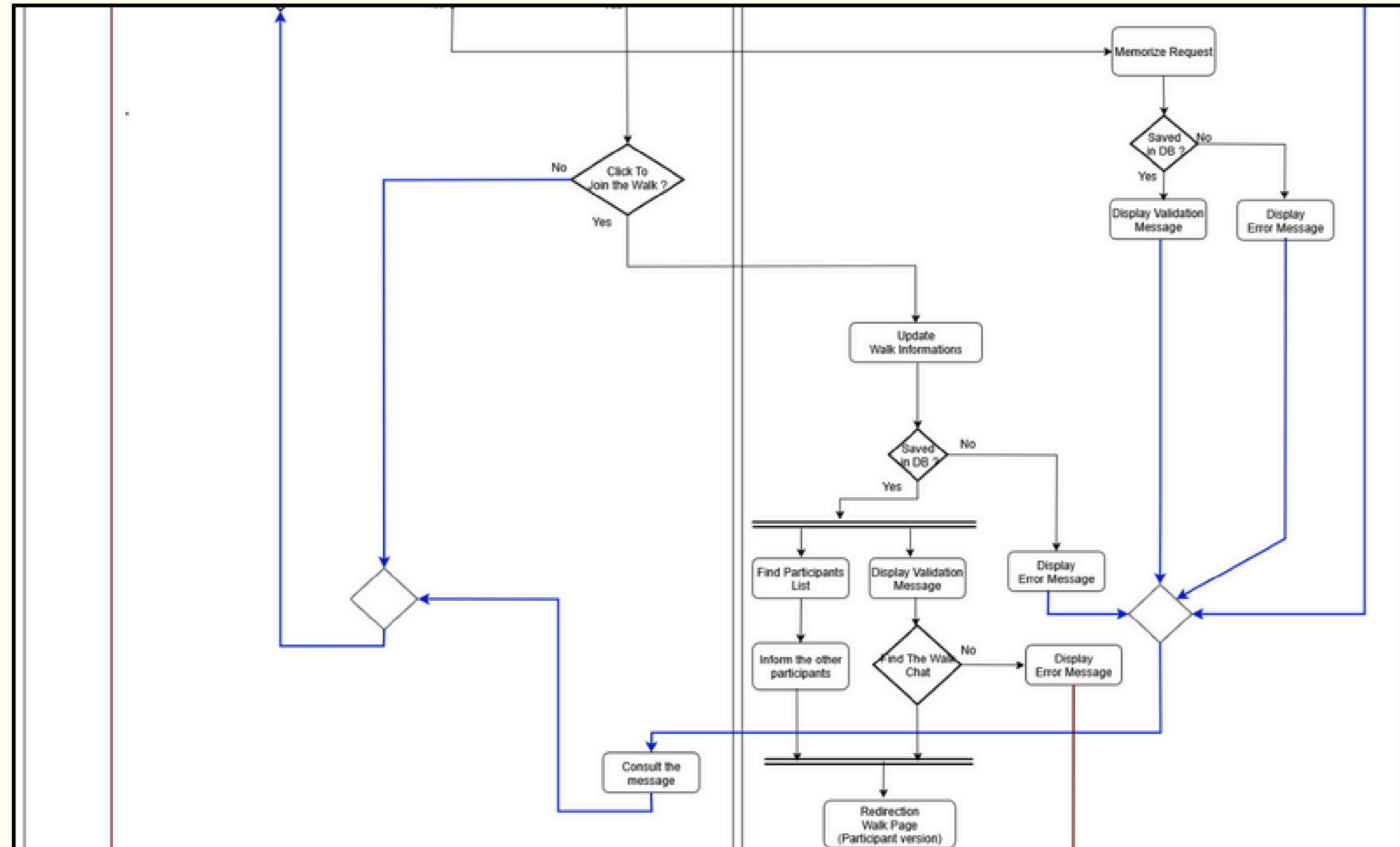
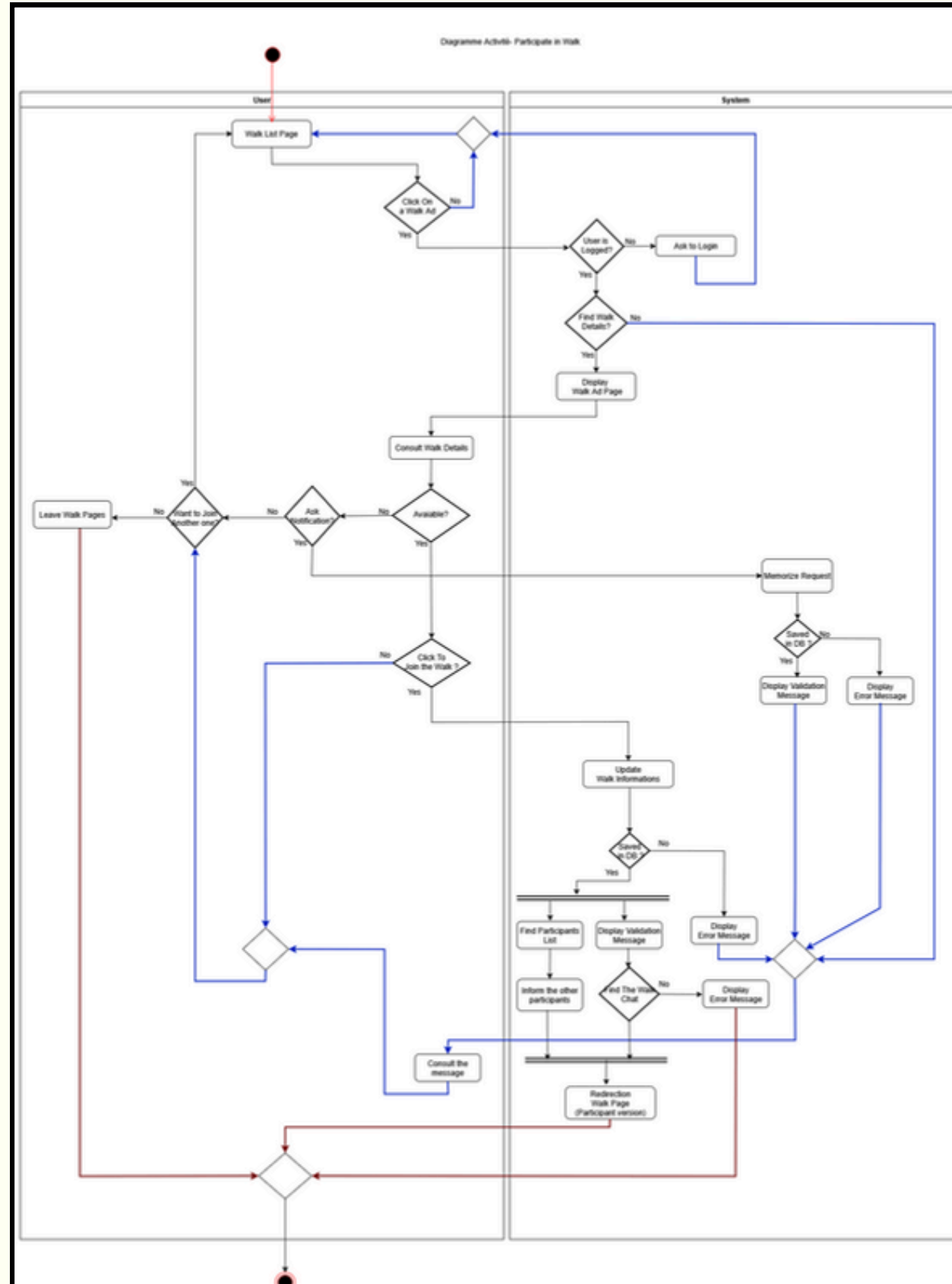
IV.11 UML : DIAGRAMME D'ACTIVITÉ INSCRIPTION BALADE

1/2



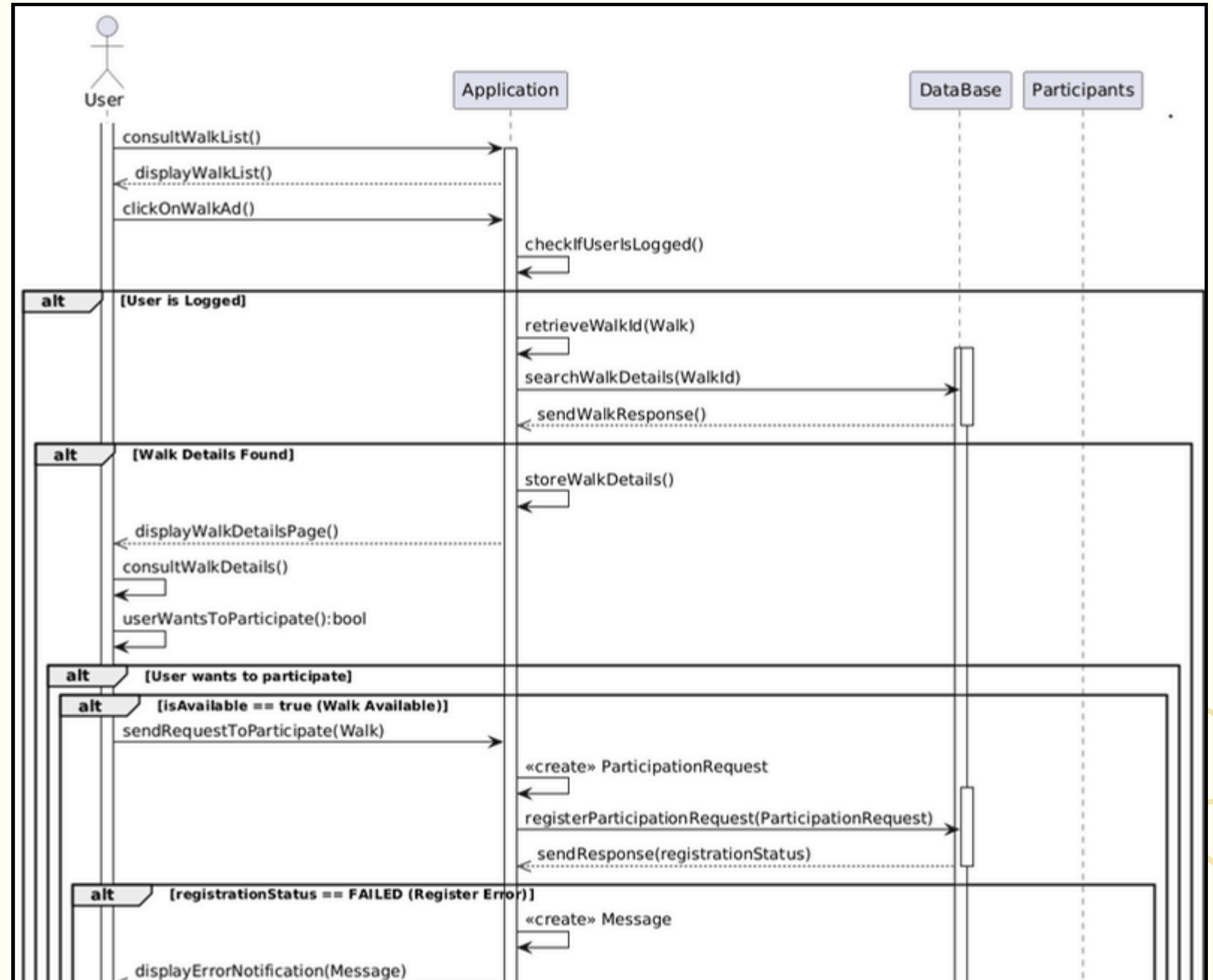
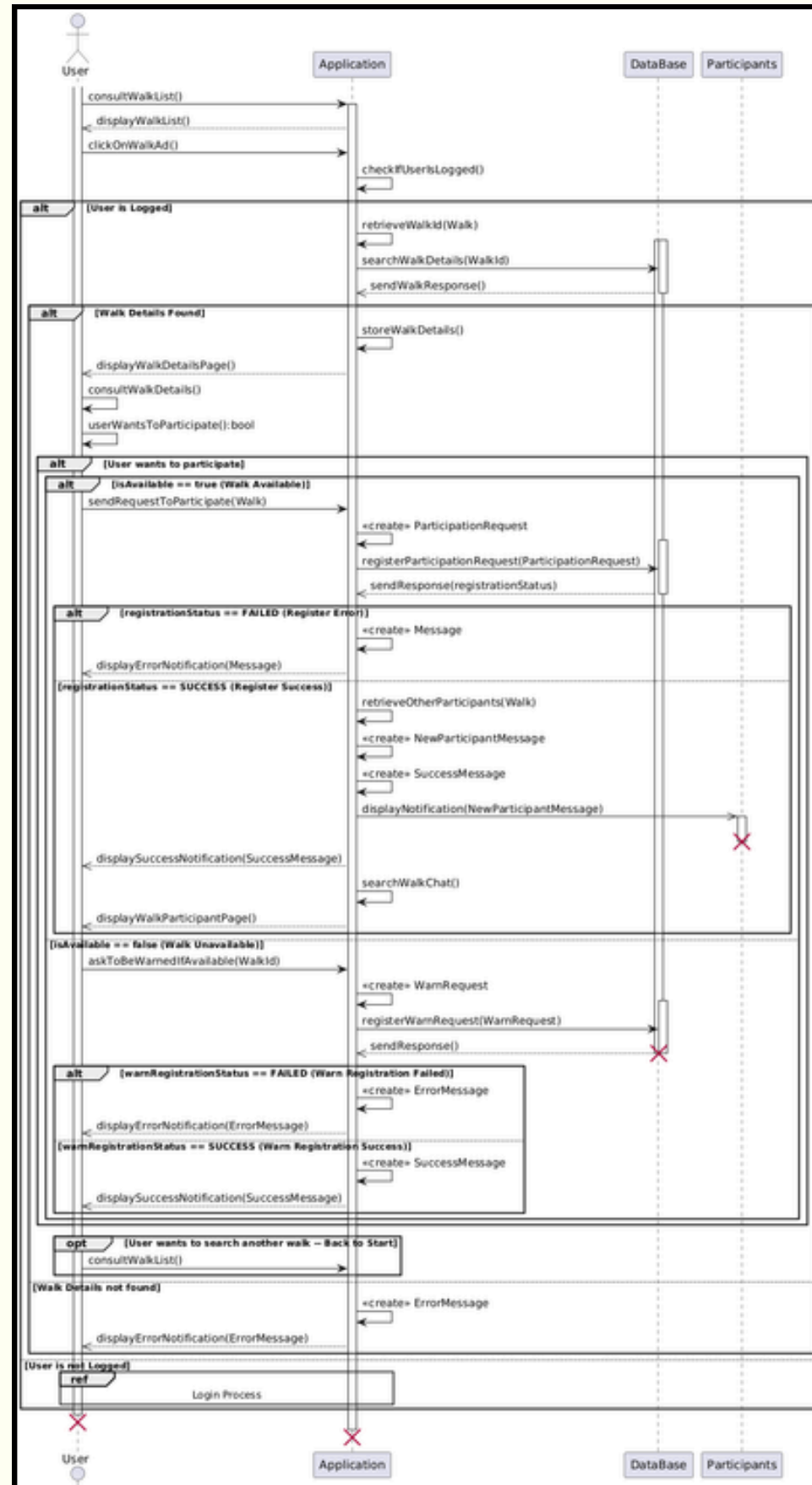
IV.11 UML : DIAGRAMME D'ACTIVITÉ INSCRIPTION BALADE

2/2

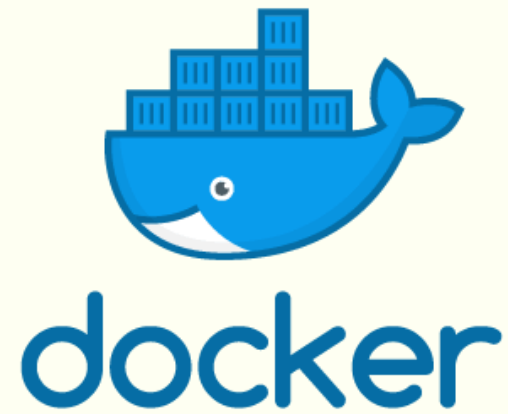
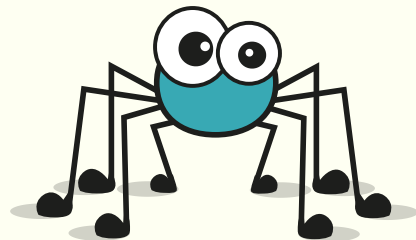
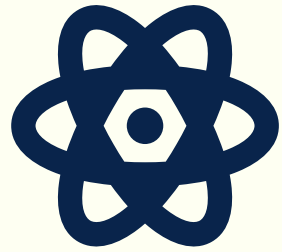




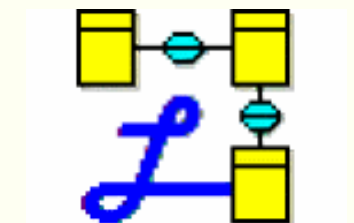
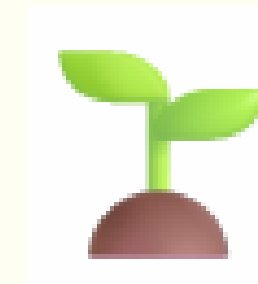
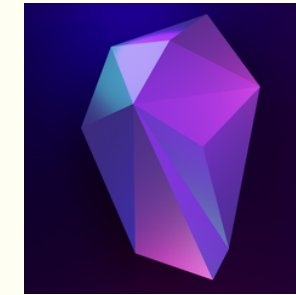
IV.12 UML : DIAGRAMME SÉQUENCE INSCRIPTION À UNE BALADE



IV.13 ENVIRONNEMENT DE TRAVAIL ET CHOIX TECHNOLOGIQUES

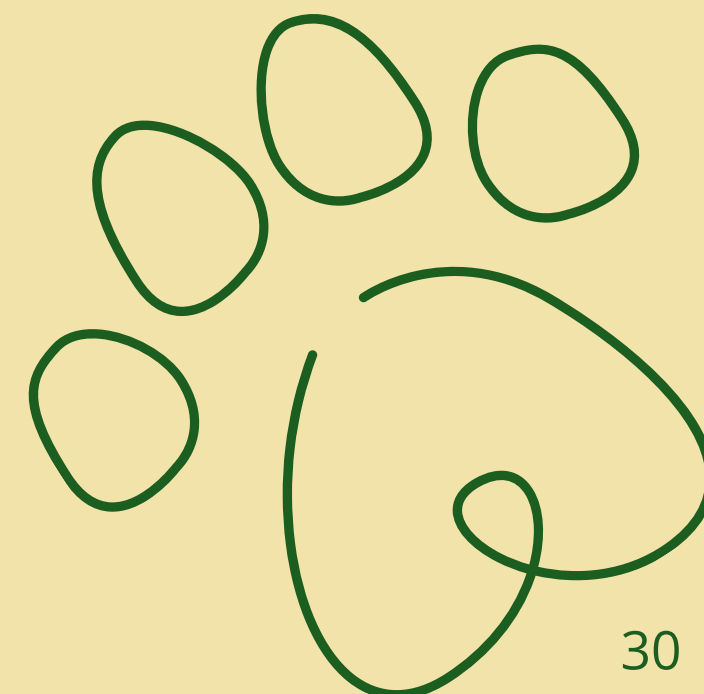


- Php fpm
- Nginx
- Frontend
- DB : BDD
- PHPMyAdmin
- Bridge network





V. CAPTURES D'ECRANS ET EXTRAITS DE CODE



V.1 POST D'UNE ANNONCE - INTERFACE



WOOFWALKS

Postez votre annonce



Titre:

Le titre est requis

Description:

La description est requise

Date et heure:

10/10/2032 22:22

Nombre maximum de participants:

Nombre maximum de participants est requis

Type de lieu:

☐ Lieu personnalisé

☒ Parc existant

Choisir un parc

-- Sélectionner un parc --

Veuillez choisir un parc

Photo: Parcourir... plus.png

Créer walks

✓ Walks créé avec succès !

WOOFWALKS





V.2 POST D'UNE ANNONCE - BOUTON POST

WALKS PAGE

```
import BtnPostAd from "../../components/Buttons/BtnPostAd";
import walkSpecificFields from "../../components/FormSpecificFields/Walks/walkSpecificFields";
import WalkList from "../../components/Lists/WalkList";

const WalksPage = () => {
  //Passage du context présent
  const formContext = "walks";
  return (
    <>
      <BtnPostAd
        formContext={formContext}
        entitySpecificFields={walkSpecificFields}
      />
      <WalkList />
    </>
  );
};

export default WalksPage;
```

BOUTON DE POST

```
const BtnPostAd = ({
  formContext,
  //Par exemple champs spécifiques à walks
  entitySpecificFields,
}) => {
  const { isAuthenticated } = useAuth();
  const navigate = useNavigate();

  const [showForm, setShowForm] = useState(false);

  const handleClick = () => {
    if (!isAuthenticated) {
      navigate("/login", { replace: true });
      return;
    }
    setShowForm(true);
  };

  return (
    <>
      {showForm ? (
        <GenericPostAdForm
          entityType={formContext}
          entitySpecificFields={entitySpecificFields}
        />
      ) : (
        <button
          onClick={handleClick}
          className="button-green btn-success btn-lg shadow-1"
        >
          Postez votre annonce
        </button>
      )}
    </>
  );
};
```


V.4 POST D'UNE ANNONCE - FORMULAIRE

GENERICPOSTADFORM - TRAITEMENT

```
const handleFileChange = (e) => {
  setPhoto(e.target.files[0]);
};

const onSubmit = async (data) => {
  if (!photo) {
    alert("Veuillez sélectionner une photo !");
    return;
  }
  setIsSubmitting(true);

  // SANITIZATION
  const sanitizedData = { ...
};
// Vérif que locationData existe avant de Sanitize
if (data.locationData) { ...
}
//récupération Id de la localisation
const locationId = await getLocationId(sanitizedData);
if (!locationId) {
  alert("La location doit être spécifiée.");
  setIsSubmitting(false);
  return;
}
try {
  const photoFormData = new FormData();
  photoFormData.append("file", photo);
  const photoData = await uploadPhoto(photoFormData);
  const photoId = photoData.id;

  const formattedDateTime = new Date(sanitizedData.datetime).toISOString();

  const entityData = {
    ...sanitizedData,
    date: formattedDateTime,
    photo: photoId,
    location: locationId,
    is_custom_location: sanitizedData.use_custom_location === "custom",
  };

  await postGenericAd(entityData, entityType);

  reset({
    title: "",
    description: "",
    location: "",
    datetime: "",
    ...entitySpecificFields.initialValues,
  });
  setPhoto(null);
}
```

- RÉCUPÉRATION ID LOCALISATION
- UPLOAD DE PHOTO
- POST DE L'ANNONCE

```
//Fonction récupération de l'id de la loc crée
const getLocationId = async (data) => {
  if (data.use_custom_location === "custom") {
    const locationData = await createLocation(data.locationData);
    return parseInt(locationData["@id"].split("/").pop());
  } else if (data.use_custom_location === "park") {
    return parseInt(data.park_location_id);
  }
  return null;
};
```

```
export const uploadPhoto = async (data) => {
  // L'intercepteur gère les erreurs, donc pas besoin de try...catch ici
  const response = await api.post("/main_photo", data);
  return response.data;
};
```

```
export const postGenericAd = async (data, entityType) => {
  try {
    const response = await api.post(`${entityType}custom`, data);
    const result = response.data;

    toast.success(
      `${entityType.charAt(0).toUpperCase() + entityType.slice(1)} créé`
    );
    return result;
  } catch (error) {
    console.error("Erreur gérée par l'intercepteur.");
    throw error;
  }
};
```

GENERICPOSTADFORM - RENDU

```
return [
  <form className="post-ad-form" onSubmit={handleSubmit(onSubmit)}>
    { /* titre */ }
    <div className="form-group"> ...
    </div>
    { /* description */ }
    <div className="form-group"> ...
    </div>
    { /* affiche champs spéciaux */ }
    {entitySpecificFields.fields.map((field) => ( ...
    ))}
    { /* Pour Walks, choix entre parc et loc custom pour les locations */ }
    {entityType === "walks" ? (
      <WalkLocationSection
        locationType={locationType}
        control={control}
        register={register}
        errors={errors}
      />
    ) : (
      // localisation api gouvernement
      <Controller
        name="locationData"
        control={control}
        defaultValue={{
          city: "",
          street: "",
          latitude: null,
          longitude: null,
          name: "",
        }}
        render={({ field }) => (
          <SelectLocationForm
            value={field.value}
            onLocationDataChange={field.onChange}
          />
        )}
      />
    )}
  </form>

  <PhotoForm photo={photo} onFileChange={handleFileChange} />

  <button type="submit" className="button-green" disabled={isSubmitting}>
    {isSubmitting ? "En cours..." : `Créer ${entityType}`}
  </button>
]
```



V.4 POST D'UNE ANNONCE - TRAITEMENTS SYMFONY

CONTROLLER

```
class WalkController extends AbstractController {
    #[Route('/api/walkscustom', name: 'create_walk', methods: ['POST'])]
    public function createWalk(
        Request $request,
        WalkWithChatCreationInterface $walkCreator,
        WalkValidationService $validationService
    ): JsonResponse {
        $user = $this->getUser();
        if (!$user) {
            return new JsonResponse(['error' => 'Authentication required'], 401);
        }
        $data = json_decode($request->getContent(), true);

        // Validation centralisée par le Service validation
        $validationResult = $validationService->validateWalkData($data);

        if (!$validationResult->isValid()) {
            return new JsonResponse(['errors' => $validationResult->getErrors()], 400);
        }
        // Passe par Interface pour appeler Service de Création
        $walk = $walkCreator->createWalkWithChat($data);

        if (!$walk) {
            return new JsonResponse(['error' => 'Failed to create walk (invalid data or dependencies)'], 400);
        }

        return new JsonResponse(['message' => 'Walk created successfully'], 201);
    }
}
```

INTERFACE

```
interface WalkWithChatCreationInterface {
    public function createWalkWithChat(array $data): ?Walk;
    // Possibilité de créer createWalkOnly ou addChatToExistingWalk
}
```

ORCHESTRATOR

```
class WalkAndChatOrchestrator implements WalkWithChatCreationInterface {
    private WalkCreationService $walkCreationService;
    private ChatCreationService $chatCreationService;

    public function __construct(
        WalkCreationService $walkCreationService,
        ChatCreationService $chatCreationService
    ) {
        $this->walkCreationService = $walkCreationService;
        $this->chatCreationService = $chatCreationService;
    }

    public function createWalkWithChat(array $data): ?Walk {
        // Création de la Walk
        $walk = $this->walkCreationService->createWalk($data);

        if (!$walk) {
            return null;
        }
        // Création du Chat associé
        $this->chatCreationService->createChatForWalk($walk);

        return $walk;
    }
}
```

REDIRECTION

```
App\Service\Contract\WalkWithChatCreationInterface:
    alias: App\Service\WalkAndChatOrchestrator
```



V.4 POST D'UNE ANNONCE - TRAITEMENTS SYMFONY

SERVICES DE CREATION

```
public function createWalk(array $data): ?Walk {
    // Validation de la Data par le Service validation
    $validationResult = $this->validationService->validateWalkData($data);
    if (!$validationResult->isValid()) {
        return null;
    }

    try {
        $datetime = new \DateTime($data['datetime']);
    } catch (\Exception $e) {
        return null;
    }

    $photo = $this->photoRepository->find($data['photo']);
    if (!$photo) {
        return null;
    }

    $location = $this->locationRepository->find($data['location']);
    $isCustomLocation = isset($data['is_custom_location']) && $data['is_custom_location'] === true;
    $creator = $this->security->getUser();

    if (!$creator instanceof User) {
        return null;
    }

    $walk = new Walk();
    $walk->setTitle($data['title']);
    $walk->setDescription($data['description']);
    $walk->setMainPhoto($photo);
    $walk->setDate($datetime);
    $walk->setMaxParticipants($data['max_participants'] ?? 0);
    $walk->setLocation($location);
    $walk->setCreator($creator);
    $walk->setIsCustomLocation($isCustomLocation);
    $walk->addParticipant($creator);

    $this->entityManager->persist($walk);
    $this->entityManager->flush();

    return $walk;
}
```

```
class ChatCreationService {
    private EntityManagerInterface $entityManager;

    public function __construct(EntityManagerInterface $entityManager) {
        $this->entityManager = $entityManager;
    }

    public function createChatForWalk(Walk $walk): Chat{
        $chat = new Chat();
        $chat->setWalk($walk);

        $walk->setChat($chat);

        $this->entityManager->persist($chat);
        $this->entityManager->flush();

        return $chat;
    }
}
```





VI. ÉLÉMENTS DE SÉCURITÉ



VI.1 SÉCURITÉ OUTILS MIS EN PLACE

ELEMENTS PRINCIPAUX

MotDePasseEnClair $\xrightarrow{\text{Hash}}$ f69b2938d13eb2c24c74d9d373b7f65a96ef52b1d320786d4bbb5190ce56d35c

- INSCRIPTION



confirm-email?token=33644d68-2996-4f32-ac1c-aa0bfa1f8fc3

- GESTION DES SESSIONS



JWT courte durée et refresh_token de longue durée
Httponly (bloque le JS)
Secure (force Https)
SameSite= Lax (protege des csrf)

- GESTION RÔLES ET ACCES



security.yaml `acces_control`

Entités `'IS_AUTHENTICATED_FULLY'` `#[Groups]` `security: "is_granted('...')"` `'ROLE_ADMIN'`

TESTS FONCTIONNELS

```
$this->assertResponseStatusCodeSame(403);
```


VI.2 SÉCURITÉ OUTILS MIS EN PLACE FAILLES ET SOLUTIONS

- NELMIO SECURITY BUNDLE



protection click-jacking

X-Content-Type-Options: nosniff

forced_ssl

ATTAQUES XSS →



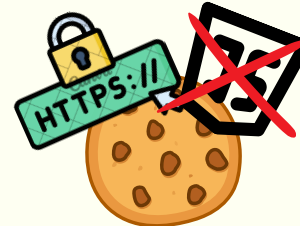
ÉLÉVATION DE PRIVILÈGES →



INJECTIONS SQL →



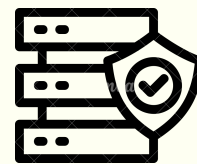
VOL DE TOKENS D'ACCÈS → JWT



HTTP-ONLY - SECURE

ATTAQUES CSRF → TOKEN CSRF

PERTE DE DONNÉES →



LOGS



FORCE BRUTE →



MOTS DE PASSE NON SÉCURISÉS →

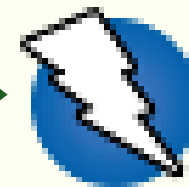
symfony/password-hasher

VOL DE DONNÉES EN TRANSIT →

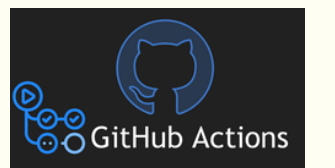


PARAMÉTRAGES DES
SECURITY HEADERS

FAILLES INCONNUES →



AUTOMATISATION TESTS →



```
#Ajout en têtes http : Empêche le navigateur de "deviner" le type de contenu d'un fichier (par exemple, d'interpréter un fichier .txt comme un script JavaScript)
add_header X-Content-Type-Options nosniff always;
#Ajoute l'en-tête Strict-Transport-Security :Force le navigateur à ne communiquer avec votre site qu'en HTTPS pendant 1 an en incluant les sous domaine
#-> défense contre attaques de l'homme au milieu : attaquant se place entre utilisateur et site web pour écouter les échanges ou modifier des données...
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
#Active le filtre XSS intégré de certains navigateurs : bloque execution de XSS si navigateur détecte
add_header X-XSS-Protection "1; mode=block" always;
#Ajoute l'en-tête de Content Security Policy : liste d'autorisation qui dit au navigateur quelles sources sont fiables pour les scripts, les styles, les images -> défense contre XSS
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' https://fonts.googleapis.com; img-src 'self' data:; font-src 'self' https://fonts.gstatic.com";
#Contrôle la mise en cache de la réponse par le navigateur : empêche stockage des infos sensibles dans le cache
add_header Cache-Control "no-store, no-cache, must-revalidate, max-age=0" always;
#Cache l'en-tête X-Powered-By de la réponse : empêche affichage de la version php
fastcgi_hide_header X-Powered-By;
# Masque la version de Nginx
server_tokens off;
```



VI.3 L'AUTH CONTEXT REACT

- ACCÈS AUX DONNÉES D'AUTHENTIFICATION, DE LOGIN ET LOGOUT PARTOUT

```
return (  
  <AuthContext.Provider value={{ isAuthenticated, isLoading, login, logout }}>  
    {isLoading ? <div>Chargement...</div> : children}  
  </AuthContext.Provider>  
);  
};
```

- FONCTION DE LOGIN/LOGOUT

```
const login = async (email, password) => {  
  await axios.post(  
    `${process.env.REACT_APP_API_URL}/api/login_check`,  
    { email, password },  
    { withCredentials: true }  
  );  
  setIsAuthenticated(true);  
};  
  
const logout = async () => {  
  console.log('tentative logout');  
  try {  
    await axios.post(`${process.env.REACT_APP_API_URL}/api/logout`, {}, { withCredentials: true });  
  } finally {  
    setIsAuthenticated(false);  
  }  
};
```

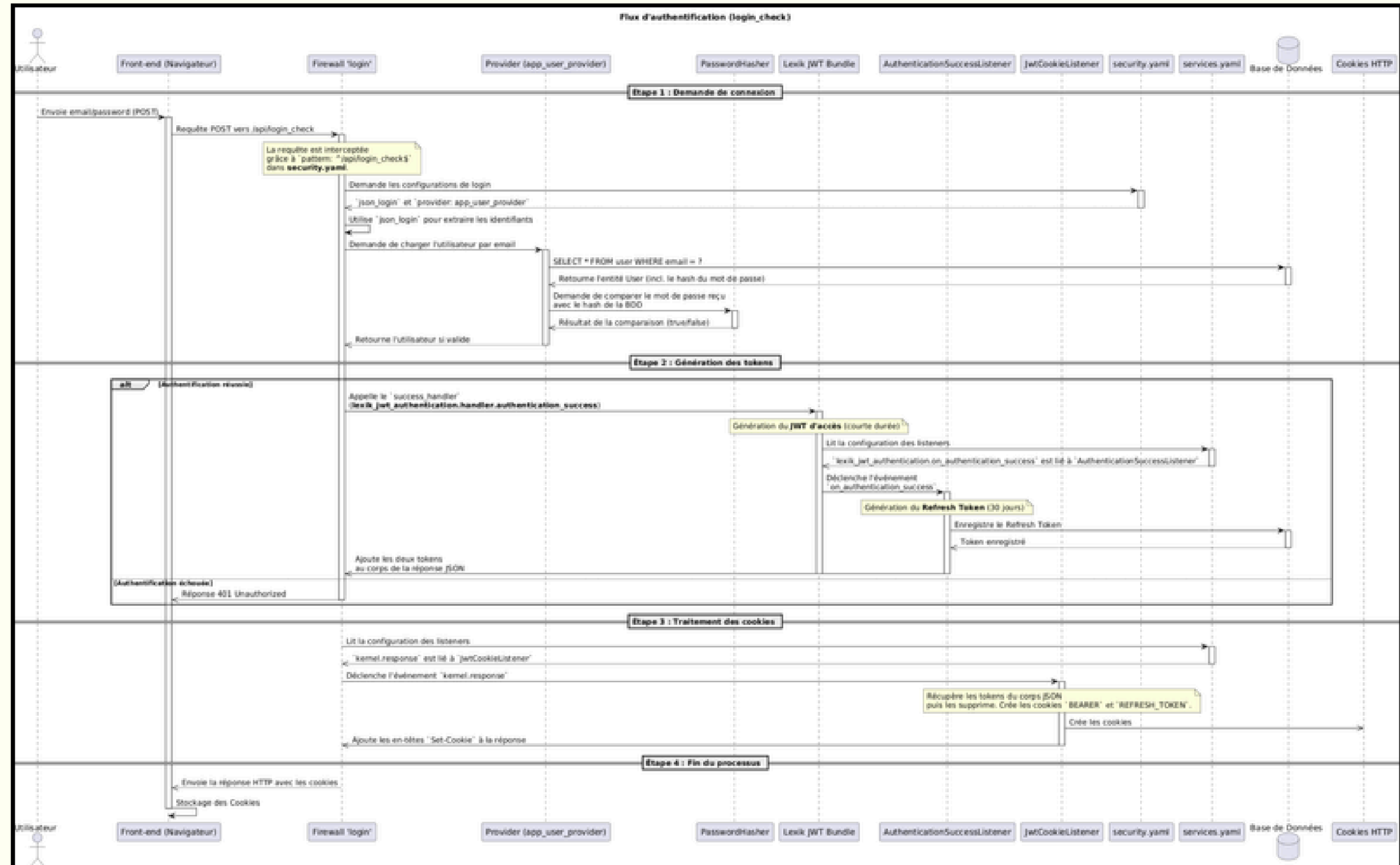
- CONTROLE SI AUTHENTIFIÉ

```
const checkAuth = useCallback(async () => {  
  try {  
    // Tente de vérifier si l'utilisateur est authentifié en appelant l'API.  
    // En cas d'erreur 401, l'intercepteur Axios tentera de rafraîchir le token.  
    await axios.get(`${process.env.REACT_APP_API_URL}/api/me`, {
```

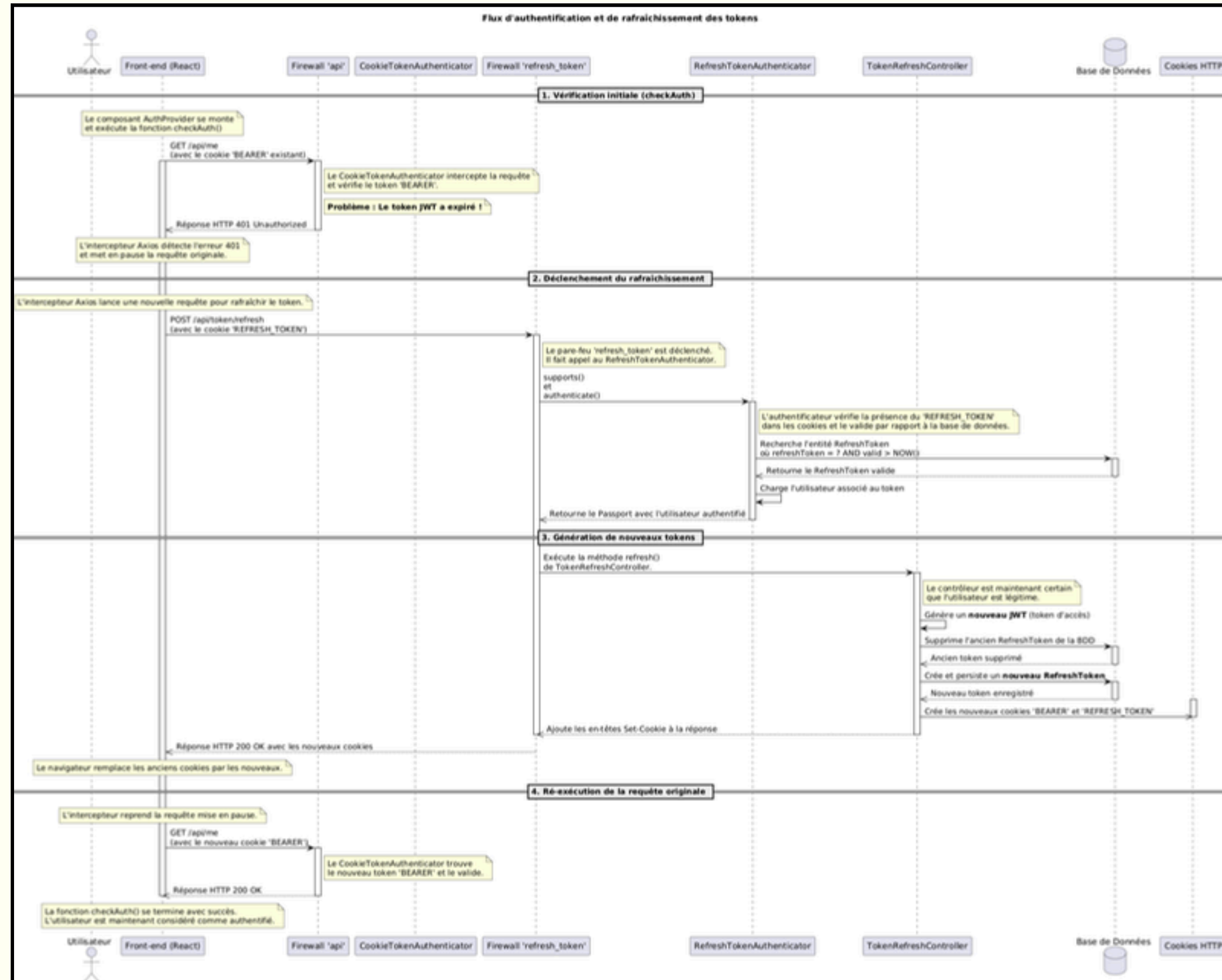
- USEEFFECT - CONFIGURE INTERCEPTEUR QUI ÉCOUTE RÉPONSE API - SI 401 : TOKEN EXPIRÉ → REFRESH

```
// Configuration de l'intercepteur Axios et de la vérification initiale de l'authentification.--gère les 401  
useEffect(() => {  
  let isRefreshing = false;  
  const responseInterceptor = axios.interceptors.response.use(  
    res => res,  
    async err => {  
      const originalRequest = err.config;  
      // Ne gère que les erreurs 401 sur les requêtes non-refresh.  
      if (err.response?.status === 401 && !originalRequest.url.endsWith('/api/token/refresh') && !isRefreshing) {  
        isRefreshing = true;  
        try {  
          console.log('Trying to refresh token...');  
          // Lance la requête de rafraîchissement.  
          await axios.post(`${process.env.REACT_APP_API_URL}/api/token/refresh`, {}, { withCredentials: true });  
          isRefreshing = false;  
          console.log('Token refreshed, retrying original request...');  
  
          // Retente la requête originale avec le nouveau token.  
          return axios({ ...originalRequest, withCredentials: true });  
        }  
      }  
    }  
  );  
}
```

VI.4 REPRÉSENTATION D'AUTHENTIFICATION



VI.5 REPRÉSENTATION DU RAFRAICHISSEMENT DU TOKEN





VII. TESTS



VII.1 PLANS DE TESTS

ID	Partie Testée	Description	Étapes	Résultat Attendu	Type de test
ECCT-002	Contrôleur Confirmation	Tenter de confirmer un email déjà vérifié.	1. Appeler la méthode confirmEmail() avec un token valide mais pour un utilisateur déjà vérifié.	Une réponse JSON avec un statut 200 et le message "Votre adresse email est déjà vérifiée."	Unitaire
ECCT-003	Contrôleur Confirmation	Confirmer un email avec succès.	1. Appeler la méthode confirmEmail() avec un token valide. - 2. Les mocks simulent un utilisateur valide.	Une réponse JSON avec un statut 200 et le message "Adresse email confirmée..."	Unitaire
WCT-001	Contrôleur Walk	Créer une balade avec succès.	1. Simuler une requête avec des données valides.-2. Le mock du service de création retourne une balade valide.	Une réponse avec un statut 201 et un message de succès.	Unitaire
WCT-002	Contrôleur Walk	Tenter de créer une balade sans tous les champs requis.	1. Simuler une requête avec des données incomplètes.	Une réponse avec un statut 400 et le message "Missing required fields".	Unitaire
WCT-003	Contrôleur Walk	Tenter de créer une balade si le service échoue.	1. Simuler une requête avec des données valides.-2. Le mock du service de création retourne null.	Une réponse avec un statut 400 et le message "Failed to create walk".	Unitaire
WCFT-001	Création de balade	Créer une balade via l'API avec succès.	1. Créer un client HTTP.-2. Générer un token JWT pour l'utilisateur.-3. Envoyer une requête POST à l'API avec les données et le token.	Un statut HTTP 201 et une réponse JSON de succès, avec le message "Walk created successfully".	Fonctionnel
JS-FT-01	User	Tenter d'accéder à la liste de tous les utilisateurs en tant qu'administrateur.	1. Créer un utilisateur avec le rôle ROLE_ADMIN. 2. Se connecter en tant que cet utilisateur. 3. Envoyer une requête GET à /api/users.	Une réponse HTTP 200 OK (succès) et un contenu JSON valide.	Fonctionnel
JS-FT-02	User	Tenter d'accéder à la liste de tous les utilisateurs en tant qu'utilisateur normal.	1. Créer un utilisateur normal. 2. Se connecter en tant que cet utilisateur. 3. Envoyer une requête GET à /api/users.	Une réponse HTTP 403 Forbidden (accès refusé).	Fonctionnel
JS-FT-03	User	Tenter d'accéder à son propre profil en tant qu'utilisateur normal.	1. Créer un utilisateur normal. 2. Générer un token JWT pour cet utilisateur. 3. Envoyer une requête GET à /api/users/{id} en utilisant l'ID de l'utilisateur, et inclure le token dans l'en-tête Authorization.	Une réponse HTTP 200 OK (succès).	Fonctionnel
JS-FT-04	User	Tenter d'accéder au profil d'un autre utilisateur en tant qu'utilisateur normal.	1. Créer deux utilisateurs normaux (user1 et user2). 2. Générer un token JWT pour user1. 3. Envoyer une requête GET à /api/users/{id} en utilisant l'ID de user2, et inclure le token de user1 dans l'en-tête Authorization.	Une réponse HTTP 403 Forbidden (accès refusé).	Fonctionnel

VII.2 JEU D'ESSAI

DESCRIPTION DES SCÉNARIOS

Jeu d'essai	Description	Résultat attendu
US-FT-01	Accès à la liste des utilisateurs en tant qu'administrateur.	Le système doit retourner un code de statut 200 OK et un contenu JSON valide.
US-FT-02	Accès à la liste des utilisateurs en tant qu'utilisateur normal.	Le système doit refuser l'accès et retourner un code de statut 403 Forbidden.
US-FT-03	Accès à son propre profil en tant qu'utilisateur normal.	Le système doit retourner un code de statut 200 OK.
US-FT-04	Accès au profil d'un autre utilisateur en tant qu'utilisateur normal.	Le système doit refuser l'accès et retourner un code de statut 403 Forbidden.

RÉSULTATS DES TESTS

Jeu d'essai	Résultat attendu	Résultat obtenu
US-FT-01	Réponse HTTP 200 OK	Réussi. La requête a bien retourné un statut 200 avec la liste de tous les utilisateurs.
US-FT-02	Réponse HTTP 403 Forbidden	Réussi. La requête a été correctement refusée avec un statut 403.
US-FT-03	Réponse HTTP 200 OK	Réussi. La requête a bien retourné un statut 200 et les données du profil de l'utilisateur authentifié.
US-FT-04	Réponse HTTP 403 Forbidden	Réussi. La requête a été correctement refusée avec un statut 403.



VII.3 IMPLÉMENTATION DE TESTS

```
public function testAdminCanSeeAllUsers(): void {
    // Création d'un admin et d'utilisateurs normaux
    $admin = $this->createUser('admin_'.uniqid().'@example.com', ['ROLE_ADMIN']);
    $user = $this->createUser('user_'.uniqid().'@example.com');
    $user = $this->createUser('user_'.uniqid().'@example.com');

    // Forcer la connexion de l'admin
    $this->client->loginUser($admin);

    // Faire la requête protégée en HTTPS
    $this->client->request(
        'GET',
        '/api/users',
        [],
        [],
        ['HTTP_ACCEPT' => 'application/json', 'HTTPS' => true]
    );

    $this->assertResponseIsSuccessful();
    $this->assertJson($this->client->getResponse()->getContent());
}

public function testRegularUserCannotSeeAllUsers(): void {
    $user = $this->createUser('user_'.uniqid().'@example.com');

    $this->client->loginUser($user);

    $this->client->request(
        'GET',
        '/api/users',
        [],
        [],
        ['HTTP_ACCEPT' => 'application/json', 'HTTPS' => true]
    );

    // Accès refusé pour un user normal
    $this->assertResponseStatusCodeSame(403);
}
```

```
public function testUserCanSeeOwnProfile(): void {

    $user = $this->createUser('user_'.uniqid().'@example.com');

    $this->client->loginUser($user);

    $jwtManager = self::getContainer()->get('lexik_jwt_authentication.jwt_manager');

    // Génère le token pour l'utilisateur
    $token = $jwtManager->create($user);

    $this->client->request(
        'GET',
        '/api/users/' . $user->getId(),
        [],
        [
            'HTTP_ACCEPT' => 'application/json',
            'HTTPS' => true,
            'HTTP_AUTHORIZATION' => 'Bearer ' . $token
        ]
    );

    $this->assertResponseIsSuccessful();
}

public function testUserCannotSeeOtherProfile(): void {

    $user1 = $this->createUser('user_'.uniqid().'@example.com');
    $user2 = $this->createUser('user_'.uniqid().'@example.com');

    $this->client->loginUser($user1);

    $jwtManager = self::getContainer()->get('lexik_jwt_authentication.jwt_manager');

    // Génère le token pour l'utilisateur
    $token = $jwtManager->create($user1);

    $this->client->request(
        'GET',
        '/api/users/' . $user2->getId(),
        [],
        [
            'HTTP_ACCEPT' => 'application/json',
            'HTTPS' => true,
            'HTTP_AUTHORIZATION' => 'Bearer ' . $token
        ]
    );

    $this->assertResponseStatusCodeSame(403);
}
```

PHPUnit 11.5.27 by Sebastian Bergmann and contributors.

Runtime: PHP 8.2.28
Configuration: /var/www/html/phpunit.dist.xml

```
...
Status : 201
Contenu : {
    "message": "Walk created successfully"
}
```

```
Status : 201
Contenu : {"message":"Walk created successfully"}
.....
```

20 / 20 (100%)

Time: 00:00.786, Memory: 44.50 MB

OK (20 tests, 67 assertions)



VIII. VEILLE





VIII. VEILLE





IX. SYNTHÈSE ET CONCLUSION





REMERCIEMENTS





MERCI...

